

Lecture 3 - Static Analysis I (BMC + SMT)

Mục lục

Lecture 3: Static Analysis I (BMC + SMT chi tiết)	2
Bài 1.6 đã dạy gì và tại sao ta cần đi tiếp	2
Lộ trình cụm	3
Mối quan hệ với Lecture 4	3
Trước khi bắt đầu	3
3.2 Verification và Validation	4
Tại sao bài này đứng trước SAT, SMT?	4
Verification và Validation: hai khái niệm rất dễ nhầm	4
Static và Dynamic Verification	5
V-model: bản đồ vị trí của V&V trong SDLC	5
V&V planning: khi nào làm gì?	7
Debugging: quá trình sau khi tool tìm bug	7
Tổng kết	8
Mini-quiz	8
3.3 State space exploration: BFS, DFS và state explosion	10
Tại sao bài này quan trọng?	10
Chương trình như một đồ thị state	10
Breadth-First Search (BFS)	11
Depth-First Search (DFS)	12
State explosion problem	13
Symbolic execution: cầu nối giữa BFS/DFS và BMC	13
DFS vs BFS với cycle: visited set quan trọng	14
Tóm tắt và chuyển tiếp	14
Mini-quiz	15
3.4 SAT và thuật toán DPLL	16
Vì sao SAT quan trọng đến vậy?	16
Bài toán SAT chính thức	16
Thuật toán DPLL: nền tảng	17
Ví dụ chạy DPLL từng bước	18
Conflict-Driven Clause Learning (CDCL)	19
Vì sao CDCL nhanh đến vậy trong thực tế?	20
Mở rộng: SAT enabling cho các ngành khác	20
Tóm tắt	20
Mini-quiz	21
3.5 SMT theories: Khi SAT không đủ	22

Vì sao SAT chưa đủ?	22
Các theory phổ biến	22
Kiến trúc DPLL(T): cốt lõi SMT solver hiện đại	24
Nelson-Oppen: ghép nhiều theory	25
Ví dụ chạy DPLL(T) từng bước	26
So sánh các SMT solver	26
Tóm tắt	27
Mini-quiz	27
3.6 Encoding numbers và floating-point	28
Vì sao bài này quan trọng?	28
Phần 1: Số nguyên với bitvector	28
Phần 2: Floating-point với IEEE 754	31
Tại sao tool BMC mặc định không dùng FP?	33
Tóm tắt	33
Mini-quiz	33
3.7 Encoding pointers và memory: phần khó nhất	35
Vì sao memory khó?	35
Phần 1: Array theory để model memory	35
Phần 2: Encoding pointer	36
Phần 3: Malloc và động học bộ nhớ	37
Phần 4: Aliasing analysis	38
Phần 5: Alignment và padding	39
Phần 6: ESBMC memory model trong thực tế	39
Tóm tắt	40
Mini-quiz	40

Lecture 3: Static Analysis I (BMC + SMT chi tiết)

Tóm tắt một dòng: Cụm này đi sâu vào cách dịch một chương trình C thành công thức logic và để SMT solver kiểm tra. Bạn sẽ thấy cách CBMC và ESBMC thực sự hoạt động bên trong, từ thuật toán SAT/DPLL ở tầng thấp cho tới encoding pointer ở tầng cao.

Bài 1.6 đã dạy gì và tại sao ta cần đi tiếp

Ở **bài 1.6**, chúng ta đã có một cái nhìn tổng quát về Bounded Model Checking: dịch chương trình sang SSA, encode thành SMT formula, đẩy cho solver, đọc kết quả. Tuy nhiên có rất nhiều thứ ta đã bỏ qua hoặc giấu dưới lớp trừu tượng “solver”. Chẳng hạn:

- SMT solver thực sự làm gì bên trong? Khi nó gặp một công thức bitvector 32-bit có hàng triệu biến boolean ngầm, thuật toán nào giúp nó quyết định SAT/UNSAT trong vài giây?
- Khi C cấp phát `malloc(100)` rồi truy cập `*(p + 50)`, làm thế nào encode được “memory” thành một đối tượng mà solver hiểu?
- Số `0.1 + 0.2` trong C trả về `0.30000000000000004` thay vì `0.3`. Làm thế nào encode chính xác semantics của floating-point để verifier không miss bug?
- Khi chương trình có nhiều file, nhiều function, làm thế nào BMC ghép hết các function lại để verify một property global?

Mỗi câu hỏi trên là một bài trong cụm này.

Lộ trình cụm

Cụm Lecture 3 sẽ đi qua bảy chủ đề chính theo thứ tự logic:

Bài	Chủ đề	Tại sao cần
3.1	Tổng quan (bài này)	Đặt context và kết nối với Lecture 1-2
3.2	Verification và Validation	Phân biệt rõ V&V, vị trí của BMC trong V-model
3.3	State space exploration	Hai chiến lược cốt lõi: BFS và DFS, cùng giới hạn của explicit-state
3.4	SAT và DPLL	Thuật toán mà SMT solver gọi ở tầng thấp nhất
3.5	SMT theories	Cách combining SAT với theory cho số, mảng, hàm
3.6	Encoding numbers và floats	Phần thực dụng nhất: dịch int, float, double C sang bitvector và FP theory
3.7	Encoding pointers và memory	Phần khó nhất: dịch *p, malloc, &x[i] sang array theory

Sau cụm này, bạn sẽ đủ kiến thức để:

- Chạy CBMC hoặc ESBMC trên code C của mình và **hiểu output** (không chỉ thấy “PASS” hoặc “FAIL”).
- Đọc paper trong lĩnh vực formal verification mà không bị choáng với các thuật ngữ “DPLL(T)”, “Nelson-Oppen”, “Lazy SMT”.
- Đánh giá khi nào dùng theory nào để cân bằng giữa tốc độ verification và độ chính xác.

Mối quan hệ với Lecture 4

Cụm Lecture 3 này tập trung **chương trình tuần tự**. Khi chương trình có nhiều thread, không gian state nhân lên theo số interleaving khả dĩ giữa các thread, và state explosion trở thành vấn đề nghiêm trọng hơn nhiều. **Lecture 4** sẽ giới thiệu các kỹ thuật chuyên biệt cho concurrency: context-bounded analysis, lazy exploration, sequentialization.

Lecture 4 giả định bạn đã hiểu rõ Lecture 3, vì các kỹ thuật concurrency phần lớn là **mở rộng** của các kỹ thuật tuần tự, không phải thay thế.

Trước khi bắt đầu

Nếu bạn chưa thoải mái với các khái niệm dưới đây, hãy quay lại các bài liên quan của Lecture 1-2 trước:

- **Property, safety vs liveness**: bài 1.5.
- **Soundness, completeness**: bài 1.5.
- **SSA form, encoding một chương trình C đơn giản**: bài 1.6.
- **Khái niệm SAT, SMT, theory**: bài 1.6.

Sẵn sàng rồi? Bắt đầu với **bài 3.2: Verification và Validation**.

3.2 Verification và Validation

Tóm tắt một dòng: Verification trả lời “Có đang xây sản phẩm đúng cách không?” (đúng đặc tả), Validation trả lời “Có đang xây đúng sản phẩm không?” (đáp ứng nhu cầu thực tế). Hai khái niệm khác nhau, dùng kỹ thuật khác nhau, và đều cần thiết.

Tại sao bài này đứng trước SAT, SMT?

Trước khi đi vào thuật toán cụ thể, chúng ta cần định vị BMC trong bức tranh tổng thể của software engineering. Nếu bạn không hiểu BMC “đứng ở đâu” trong quy trình phát triển phần mềm, bạn sẽ khó biết khi nào dùng nó, và quan trọng hơn, **khi nào không cần dùng**.

Bài này trả lời ba câu hỏi:

1. Sự khác biệt giữa Verification và Validation là gì?
2. V-model là gì, và BMC nằm ở đâu trong V-model?
3. Khi tool tìm thấy bug, quy trình debug tiếp theo trông như thế nào?

Verification và Validation: hai khái niệm rất dễ nhầm

Một câu nói rất hay của Barry Boehm (cha đẻ của mô hình spiral):

“Verification: Are we building the product right?” “Validation: Are we building the right product?”

Cùng dùng chữ “right” nhưng nghĩa khác hẳn nhau. Hãy phân tích từng cái.

Verification (kiểm chứng) hỏi: chương trình có làm đúng những gì đặc tả yêu cầu không? Giả sử đặc tả nói “hàm $\text{sqrt}(x)$ trả về số y thỏa $y \cdot y \approx x$ với sai số dưới 10^{-9} ”. Verification kiểm tra implementation của sqrt có thực sự đạt sai số đó không. Câu trả lời chỉ phụ thuộc vào hai thứ: đặc tả và code. Không cần biết “có ai dùng hàm này hay không”, “có ích cho user không”.

Validation (xác nhận) hỏi: cái mà ta đang xây có giải quyết đúng vấn đề mà người dùng cần không? Có khi đặc tả nói “tính sqrt ” rất chính xác, nhưng người dùng thực ra cần $\log$ chứ không phải sqrt . Implementation đúng đặc tả nhưng không giải quyết đúng nhu cầu. Validation kiểm tra “tầng cao hơn”: đặc tả có đúng với mục tiêu kinh doanh, có đúng với user story, có đúng với pháp luật và quy định.

Để dễ nhớ, đây là một phép loại suy quen thuộc: bạn order một chiếc bánh sinh nhật. Bạn nói với thợ làm bánh “tôi muốn bánh kem socola, hình tròn, đường kính 20cm”. Thợ làm bánh xem xét đặc tả, dùng đúng nguyên liệu, đúng nhiệt độ lò, đúng thời gian. Cuối cùng giao bánh đúng tất cả mọi spec đó. Đây là **verification thành công**.

Nhưng giả sử bạn quên nói “không có hạt”, và người ăn bánh dị ứng hạt. Đặc tả không đề cập, nên thợ làm bánh không sai. Tuy nhiên sản phẩm cuối **không đáp ứng nhu cầu thực**. Đây là **validation thất bại**.

Trong software:

Tiêu chí	Verification	Validation
Câu hỏi	Code đúng spec không?	Spec đúng nhu cầu user không?
Đối tượng kiểm tra	Code vs spec	Spec vs reality
Kỹ thuật điển hình	BMC, code review, unit test, type check	User acceptance test, beta test, prototype
Người thực hiện	Developer, QA	Product manager, user

Tiêu chí	Verification	Validation
Thời điểm	Suốt SDLC, đặc biệt sau khi viết code	Đầu SDLC (requirement), cuối SDLC (UAT)

:::tip[Nhớ bằng câu ngắn] **Verification** = “Viết đúng” (làm đúng spec). **Validation** = “Viết đúng” (làm đúng cái đáng làm). :::

Static và Dynamic Verification

Verification chia thành hai họ kỹ thuật chính dựa vào việc **có chạy chương trình không**.

Static verification không chạy chương trình. Tool đọc source code (hoặc bytecode, IR), phân tích nó như một đối tượng toán học. Ví dụ điển hình: - Code review (thủ công). - Type checking (compiler tự làm). - Linter (ESLint, clang-tidy). - Abstract interpretation (Astrée, Frama-C). - Bounded Model Checking (CBMC, ESBMC). - Theorem proving (Coq, Isabelle).

Ưu điểm: **không cần input để chạy**. Static analysis có thể chứng minh tính chất cho **mọi input** mà chương trình có thể nhận. Tool sound thậm chí cho ta proof là “không có bug nào trong scope đã verify”.

Nhược điểm: bị giới hạn bởi tính undecidable của đa số property (định lý Rice). Mọi tool đều phải đánh đổi giữa soundness, completeness, scalability.

Dynamic verification chạy chương trình với input cụ thể, quan sát hành vi. Ví dụ điển hình: - Unit test, integration test, e2e test. - Fuzzing (AFL, libFuzzer). - Runtime monitoring (AddressSanitizer, ThreadSanitizer). - Property-based testing (QuickCheck, Hypothesis).

Ưu điểm: **đơn giản, scale tốt**. Bạn không cần tool đặc biệt để chạy code. Bug tìm thấy là bug thật, không phải false positive.

Nhược điểm: chỉ tìm bug, không chứng minh được “không có bug”. Bị giới hạn bởi input space (đã phân tích ở [bài 1.5](#)).

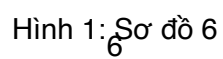
Tiêu chí	Static	Dynamic
Chạy code không?	Không	Có
Cần input không?	Không	Có
Phát hiện được “mọi bug”?	Có thể, nếu sound + complete	Không, chỉ bug có input tái tạo được
Tốc độ	Chậm hơn cho bug đơn giản	Nhanh cho bug đơn giản
False positive	Thường có	Hiếm
False negative	Tùy tool	Luôn có khả năng

Trong tài liệu này, Lecture 3-4 dạy static (BMC + SMT), Lecture 5 dạy dynamic (fuzzing, monitoring). Hai họ này **bổ sung nhau**, không thay thế nhau.

V-model: bản đồ vị trí của V&V trong SDLC

Trong các quy trình phát triển phần mềm truyền thống, V&V được tích hợp qua một mô hình tên là **V-model**. Tên gọi V đến từ hình dáng chữ V mà mô hình vẽ ra: bên trái xuống là các bước thiết kế từ trừu tượng tới chi tiết, bên phải lên là các bước kiểm tra từ chi tiết tới trừu tượng.

Mỗi tầng bên trái có một tầng kiểm tra tương ứng bên phải:



- **Requirement analysis** (yêu cầu) đối chiếu với **User Acceptance Testing (UAT)**. UAT là validation: kiểm tra sản phẩm có đáp ứng nhu cầu user không.
- **System design** đối chiếu với **System Testing**: test toàn bộ hệ thống xem có đúng design không.
- **Architecture design** đối chiếu với **Integration Testing**: test các module ghép với nhau có đúng interface không.
- **Module design** đối chiếu với **Unit Testing**: test từng function, class theo spec.

BMC (như CBMC, ESBMC) hoạt động chủ yếu ở **tầng Unit Testing**, ở chỗ ta kiểm tra một function hoặc một module có thỏa property cụ thể không. Nó là một dạng “unit test mạnh hơn” có khả năng chứng minh cho mọi input thay vì chỉ test một số input mẫu.

Mở rộng lên cao hơn, BMC còn dùng được ở **Integration Testing** khi ta gán nhiều function lại và verify property cross-function. Tuy nhiên ở mức System Testing trở lên, BMC ít dùng vì state space quá lớn.

:::warning[V-model có thực sự dùng trong agile?] V-model là khái niệm của thời waterfall. Trong agile/scrum hiện đại, các tầng V&V không tách biệt rạch ròi mà được làm song song trong từng sprint. Tuy nhiên các khái niệm “unit test”, “integration test”, “system test”, “UAT” vẫn được sử dụng, và mối quan hệ V&V với từng tầng vẫn đúng. V-model có giá trị như một bản đồ khái niệm, không phải quy trình ép buộc. :::

V&V planning: khi nào làm gì?

Khi lên kế hoạch V&V cho một dự án, ta cần trả lời các câu hỏi sau:

Property nào cần verify? Liệt kê các thuộc tính (functional, security, performance, usability) mà sản phẩm phải có. Mỗi thuộc tính cần phương pháp test riêng. Security property như “no buffer overflow” phù hợp BMC. Performance property như “throughput > 1000 req/s” cần load test. Usability property cần user testing.

Khi nào verify? Sớm tốt hơn muộn. Quy tắc Boehm nổi tiếng: chi phí sửa bug tăng theo cấp số mũ qua các giai đoạn. Bug tìm thấy lúc requirement: \$1. Lúc design: \$5. Lúc code: \$10. Lúc test: \$50. Sau production: \$200 trở lên. Vì thế:

- Code review nên làm khi vừa viết xong, trước khi merge.
- Unit test nên có cùng commit với code (TDD).
- BMC nên chạy trong CI mỗi lần PR.
- Integration test có thể làm theo lịch hằng đêm.

Ai làm? Developer nên tự test trước khi merge (smoke test). QA độc lập làm test rộng hơn. Security team làm pen test cho production system.

Tool nào? Phụ thuộc tech stack: - C/C++: CBMC, Frama-C, KLEE, AFL. - Java: Java PathFinder (JPF), JBMC, Spotbugs. - Smart contract: Certora, Manticore, Mythril. - Python: pytest + mypy + bandit.

Debugging: quá trình sau khi tool tìm bug

Khi tool báo bug, công việc của developer mới thực sự bắt đầu. Quy trình debug điển hình gồm năm bước:

Bước 1: Tái tạo bug (reproduce). Developer chạy lại với cùng input để xác nhận bug có thật, không phải artifact của test environment. Với BMC, tool đã cung cấp counterexample (input cụ thể), bước này nhanh. Với fuzzer, có thể cần thử lại nhiều lần với cùng seed.

Bước 2: Cô lập bug (localize). Xác định module, function, dòng code gây ra bug. Bước này dùng binary search, print debug, debugger (gdb, lldb). Với BMC, tool thường chỉ rõ assertion bị vi phạm và trace dẫn

tới đó.

Bước 3: Hiểu cơ chế (understand). Tại sao bug xảy ra? Là off-by-one? Race? Type confusion? Bước này cần expertise về lớp lỗi hỏng. Tài liệu này (Lecture 1-2) đã giới thiệu các lớp phổ biến.

Bước 4: Sửa (fix). Viết patch. Quan trọng: phải hiểu **root cause** trước khi sửa. Sửa symptom có thể che bug nhưng nó vẫn còn. Ví dụ, một crash do null pointer có thể được “sửa” bằng `if (p) { ... }`, nhưng câu hỏi đúng là: tại sao p lại null? Chỗ nào đáng nhẽ phải khởi tạo nó?

Bước 5: Regression test (ngăn tái phát). Viết test case từ counterexample. Khi sửa bug, test pass. Khi ai đó vô tình tái tạo bug trong tương lai, test fail ngay. Đây là cách dự án trưởng thành tích lũy “memory” về các bug đã sửa.

:::tip[Workflow BMC + regression test] Một workflow rất hiệu quả khi dùng BMC: mỗi lần CBMC báo SAT (tìm bug), copy counterexample input ra một file `tests/counterexamples/bug_NNN.c`, biến nó thành unit test. Khi sửa bug, unit test pass. Khi merge PR, CI chạy cả CBMC (verify cho mọi input) lẫn unit test (test counterexample cũ). Hai lớp này bổ sung nhau: CBMC chống bug mới, unit test chống regression. :::

Tổng kết

- **Verification** kiểm tra code vs spec. **Validation** kiểm tra spec vs nhu cầu.
- **Static V** không chạy code, **Dynamic V** chạy code. Hai cách bổ sung nhau.
- **V-model** đặt V&V tương ứng với từng tầng SDLC. BMC chủ yếu ở tầng Unit và Integration.
- Quy trình debug: reproduce, localize, understand, fix, regression test. Counterexample từ BMC là điểm bắt đầu hoàn hảo cho regression test.

Mini-quiz

Q1. Một sản phẩm pass tất cả unit test, integration test, system test nhưng người dùng không thèm dùng. Đây là vấn đề verification hay validation?

Validation thất bại. Mọi V (verification) thành công vì code làm đúng spec. Nhưng spec không đáp ứng đúng nhu cầu user (ví dụ tính năng không quan trọng, UI khó dùng, không phù hợp văn hóa địa phương). UAT đáng nhẽ phải phát hiện trước launch.

Q2. Tại sao BMC ít dùng cho System Testing?

System Testing kiểm tra toàn bộ ứng dụng đầu cuối, bao gồm UI, database, network, external service. Mỗi tầng này đem theo state riêng. State space của cả hệ thống quá lớn (số byte trong RAM database, số connection, số file descriptor) cho BMC encode.

BMC phù hợp ở tầng Unit và Integration nơi state space của một function hoặc một module vẫn manageable. Ở tầng System, ta dùng dynamic testing (e2e test, load test, chaos engineering) thay thế.

Q3. Khi tool BMC báo SAT với một counterexample, bước 1 trong quy trình debug (reproduce) đáng nhẽ rất nhanh. Vì sao đôi khi vẫn khó tái tạo?

Có vài lý do: - BMC chạy trên model **trừu tượng** của chương trình. Counterexample có thể giả định một concrete value mà thực tế không thể đạt do environment (input từ network, file system) bị filter trước. - BMC có thể model environment không chính xác. Ví dụ giả định một stdin đọc bất kỳ byte nào, nhưng thực tế stdin chỉ chứa printable characters. - Counterexample đôi khi quá lớn để gõ thủ công (mảng 1000 phần tử). Cần tool sinh test case tự động từ counterexample.

Khi gặp tình huống “BMC SAT nhưng không reproduce được”, thường vấn đề là **environment model**: cần thêm assumption vào BMC để loại trừ counterexample không khả thi trong thực tế.

Tiếp theo: 3.3 State space exploration

3.3 State space exploration: BFS, DFS và state explosion

Tóm tắt một dòng: Mọi chương trình có thể được nhìn như một đồ thị state, kiểm chứng đồng nghĩa với khám phá đồ thị này tìm trạng thái xấu. BFS và DFS là hai chiến lược cơ bản nhất, mỗi cái có ưu nhược điểm riêng. Vấn đề lớn nhất là state explosion: số state lớn theo cấp số mũ.

Tại sao bài này quan trọng?

Khi nghe “BMC dịch chương trình sang công thức rồi solver giải”, ta dễ tưởng chỉ có một cách tiếp cận. Thực tế, BMC là **một** trong nhiều phương pháp khám phá state space của chương trình. Trước khi BMC ra đời, người ta dùng explicit-state model checking với BFS/DFS. Hiểu các phương pháp này giúp ta hiểu **vì sao BMC ra đời** và **khi nào nó vượt trội**.

Bài này trả lời:

1. Chương trình là một đồ thị state như thế nào?
2. BFS và DFS hoạt động ra sao, mỗi cái có thế mạnh gì?
3. State explosion là gì và có những kỹ thuật nào để chống?
4. Symbolic execution kết nối với BMC ra sao?

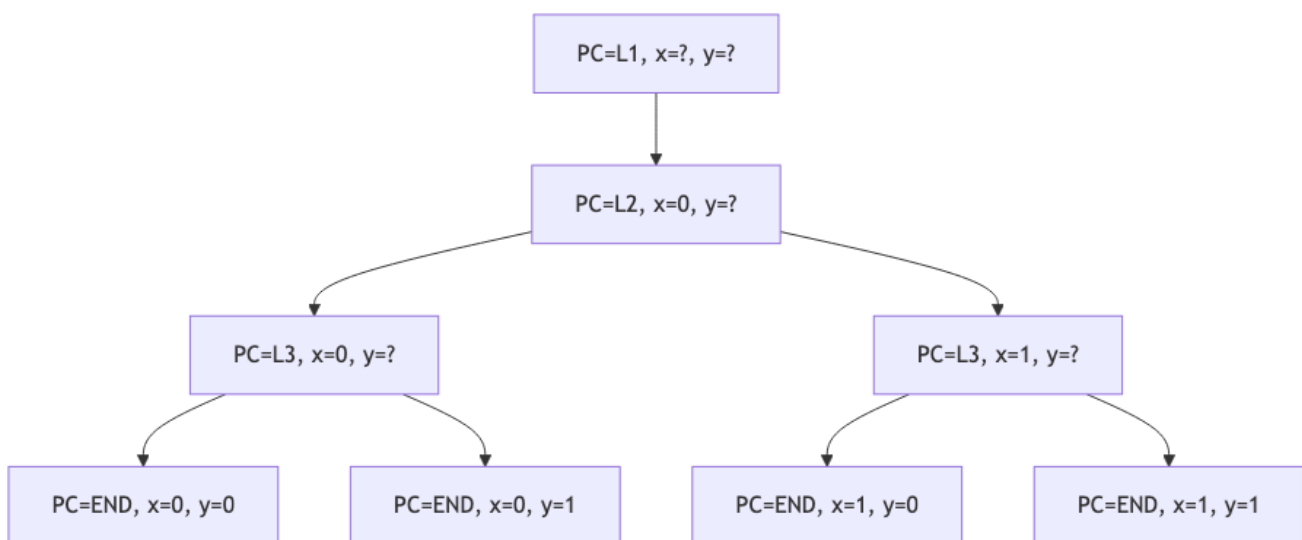
Chương trình như một đồ thị state

Hãy bắt đầu với một ví dụ rất nhỏ:

```
int x = 0;
int y = 0;
if (input1) x = 1;
if (input2) y = 1;
```

Mỗi state của chương trình tại một thời điểm có thể được mô tả bởi giá trị của hai biến x và y cùng với program counter (PC) chỉ instruction tiếp theo. Khi chương trình chạy, state thay đổi theo từng bước.

Đồ thị state cho chương trình trên (với input1 và input2 đều có thể là 0 hoặc 1):



Hình 2: Sơ đồ 7

Bốn state cuối là bốn outcome có thể xảy ra. Để verify một property, ta khám phá đồ thị này và kiểm tra mọi state cuối có thoả property không.

Quan trọng cần hiểu: số state này là 2^n nếu có n biến boolean độc lập. Chương trình thực có nhiều biến int 32-bit, dẫn tới 2^{32} giá trị mỗi biến. Tổ hợp của k biến int là 2^{32k} . Đây chính là **state explosion problem** mà ta sẽ quay lại sau.

Breadth-First Search (BFS)

BFS khám phá state theo lớp, từ gần tới xa. Bắt đầu từ initial state, thăm hết các state hàng xóm (depth 1), rồi tới hàng xóm của hàng xóm (depth 2), và cứ thế. Dùng một queue (FIFO) để lưu state đang chờ thăm.

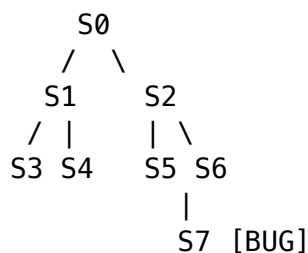
Pseudocode:

```
function BFS(s0, property):
    queue = [s0]
    visited = {s0}
    while queue not empty:
        s = queue.pop_front()
        if not property(s):
            return CounterExample(s)
        for s' in successors(s):
            if s' not in visited:
                visited.add(s')
                queue.push_back(s')
    return Proof
```

Thế mạnh chính của BFS: khi tìm thấy counterexample, đó là counterexample **ngắn nhất** (ít transition nhất từ initial state). Điều này cực kỳ có ích cho debug: developer thấy được “chuỗi event tối thiểu” dẫn tới bug, dễ hiểu hơn nhiều so với một trace dài lê thê.

Ví dụ BFS minh hoạ

Xét đồ thị state đơn giản (8 state, các transition như mũi tên):



BFS từ S0 sẽ thăm theo thứ tự: S0, S1, S2, S3, S4, S5, S6, S7. Khi tới S7, phát hiện bug. Trace counterexample: S0 → S2 → S5 → S7 (3 bước).

DFS có thể thăm: S0, S1, S3, S4, S2, S5, S7. Trace counterexample cũng dẫn tới S7 nhưng phải khám phá nhiều state hơn trước khi tới đó (vì DFS đi sâu vào nhánh S1 trước).

Nhược điểm BFS

BFS phải **lưu mọi state ở mỗi lớp** trong queue. Với đồ thị có branching factor b và depth d , queue có thể chứa b^d state. Memory tăng nhanh, dẫn tới out-of-memory ngay cả với chương trình nhỏ.

Depth-First Search (DFS)

DFS khám phá theo nhánh, đi sâu nhất có thể rồi backtrack. Dùng một stack (LIFO) thay queue.

Pseudocode:

```
function DFS(s0, property):
    stack = [s0]
    visited = {s0}
    while stack not empty:
        s = stack.pop()
        if not property(s):
            return CounterExample(s)
        for s' in successors(s):
            if s' not in visited:
                visited.add(s')
                stack.push(s')
    return Proof
```

Bản chất DFS rất giống cách con người duyệt cây: đi xuống một nhánh tới khi gặp lá, quay lui, thử nhánh khác.

Thế mạnh DFS

Memory của DFS chỉ là $O(d)$ (chiều sâu), tốt hơn BFS rất nhiều. Vì thế DFS scale tới depth lớn hơn.

DFS cũng tự nhiên với **symbolic execution**: mỗi nhánh trong DFS tương ứng với một path qua chương trình. Khi DFS đi sâu vào một nhánh, nó implicit “chọn” một path, và có thể tính path constraint dọc đường.

Nhược điểm DFS

- Counterexample tìm được có thể dài hơn cần thiết (không tối ưu).
- Trong đồ thị có cycle, DFS có thể đi vào loop nếu không có visited check.
- Khó parallel hoá so với BFS (mỗi nhánh DFS phụ thuộc nhánh trước).

Khi nào dùng cái nào?

Tiêu chí	BFS phù hợp khi	DFS phù hợp khi
Memory	Đủ RAM	Chật RAM
Counterexample ngắn nhất	Cần	Không quan trọng
Depth lớn, branching nhỏ	Tốn RAM	Phù hợp
Depth nhỏ, branching lớn	Phù hợp	Có thể bỏ sót sớm
Parallel?	Dễ	Khó

Trong thực tế, model checker như SPIN dùng DFS làm default vì memory là bottleneck chính. Tool như UPPAAL dùng kết hợp tùy scenario.

State explosion problem

Đây là vấn đề trung tâm của model checking. Hãy hình dung một ví dụ cụ thể.

Một chương trình có: - 10 biến boolean. - 5 biến int 8-bit (giá trị 0 đến 255). - Một mảng `arr[16]` chứa byte.

Số state khả dĩ là $2^{10} \times 256^5 \times 256^{16} = 2^{10} \times 2^{40} \times 2^{128} = 2^{178}$.

Số nguyên tử trong vũ trụ quan sát được vào khoảng $10^{80} \approx 2^{266}$. Nghĩa là state space của một chương trình nhỏ đã lớn cỡ một phần nhỏ của vũ trụ. Không có máy tính nào liệt kê hết được.

Hơn nữa, khi chương trình có k thread, state explosion nhân lên: mỗi thread có vị trí PC riêng, và mọi interleaving giữa các thread đều là state riêng. Số interleaving của k thread, mỗi thread n instruction, là $\frac{(kn)!}{(n!)^k}$. Với $k = 4, n = 10$ đã là $\approx 10^{19}$. Đây là chủ đề chính của **Lecture 4**.

Các kỹ thuật chống state explosion

Có ba họ kỹ thuật chính:

Symbolic representation thay vì liệt kê từng state, biểu diễn cả tập state bằng một công thức. Ví dụ thay vì lưu 256 state $\{x = 0, x = 1, \dots, x = 255\}$, lưu công thức $0 \leq x \leq 255$. Đây là nền tảng của symbolic model checking và BMC.

Partial-order reduction với chương trình concurrent: nếu hai instruction từ hai thread giao hoán (không ảnh hưởng nhau), chỉ cần khám phá một thứ tự, không cần cả hai. Giảm số interleaving phải khám phá.

Abstract interpretation thay vì giá trị chính xác, dùng abstract domain (interval, polyhedra) over-approximate. Số abstract value nhỏ hơn nhiều, không bị explosion.

Symbolic execution: cầu nối giữa BFS/DFS và BMC

Trước khi đi vào SAT/SMT chi tiết, ta nên hiểu **symbolic execution** vì nó là cầu nối tự nhiên từ DFS truyền thống sang BMC.

Ý tưởng: thay vì chạy chương trình với input cụ thể, chạy với input **symbolic** (biến chưa biết giá trị). Khi gặp `if (x > 5)`, ta không quyết định nhánh nào ngay, mà **fork** thành hai world: một world giả định $x > 5$, một world giả định $x \leq 5$. Mỗi world tích lũy **path constraint** dọc đường.

Ví dụ:

```
int abs(int x) {
    if (x >= 0)
        return x;
    else
        return -x;
}
```

Symbolic execution với input x symbolic:

- Path 1: giả định $x \geq 0$. Path constraint: $x \geq 0$. Return value: x .
- Path 2: giả định $x < 0$. Path constraint: $x < 0$. Return value: $-x$.

Nếu ta muốn check property “return value ≥ 0 ”, với mỗi path: - Path 1: cần $x \geq 0 \wedge x < 0$. Solver trả UNSAT, không vi phạm property. - Path 2: cần $x < 0 \wedge -x < 0$, tức $x < 0 \wedge x > 0$. UNSAT.

Cả hai path đều OK, property chứng minh được.

Nhưng nếu encode bitvector 32-bit, path 2 cần $x < 0 \wedge -x < 0$. Trong bitvector, $x = \text{INT_MIN}$ cho $-x = \text{INT_MIN}$ (overflow). Solver tìm được model $x = -2^{31}$, vi phạm property. Counterexample này chính xác lại là cái mà BMC tìm được ở bài 1.6.

Symbolic execution và BMC rất gần nhau về bản chất. Sự khác biệt:

Tiêu chí	Symbolic Execution	BMC
Khám phá theo	Path (forward)	Encode toàn bộ thành 1 công thức
Số gọi solver	Nhiều (1 lần / path)	1 lần (cho toàn formula)
Path explosion	Vấn đề chính	Ẩn trong solver
Tool	KLEE, SAGE, S2E	CBMC, ESBMC

KLEE là tool symbolic execution nổi tiếng nhất, dùng để tìm bug trong coreutils, libc, kernel driver.

DFS vs BFS với cycle: visited set quan trọng

Một điểm thực dụng cần lưu ý: trong đồ thị state có cycle (ví dụ chương trình có loop), nếu DFS không check visited, nó sẽ đi vào loop vô hạn.

```
def dfs_buggy(s):
    if not property(s):
        return CounterExample(s)
    for s_next in successors(s):
        dfs_buggy(s_next)    # KHÔNG check visited!
```

Phiên bản đúng:

```
def dfs_correct(s, visited):
    if s in visited:
        return
    visited.add(s)
    if not property(s):
        return CounterExample(s)
    for s_next in successors(s):
        dfs_correct(s_next, visited)
```

Trong chương trình thực, “state” có thể là tổ hợp giá trị mọi biến + PC, rất khó hash. Tool model checker dùng các kỹ thuật như Bloom filter, BDD canonical form để check visited hiệu quả.

Tóm tắt và chuyển tiếp

- **State space** của chương trình là đồ thị; verify là khám phá đồ thị tìm bad state.
- **BFS** tìm counterexample ngắn nhất nhưng tốn RAM. **DFS** tiết kiệm RAM nhưng counterexample dài.
- **State explosion** là vấn đề trung tâm: số state lớn theo cấp số mũ với số biến.
- **Symbolic representation** (cụm BMC + SMT) là cách giảm state explosion hiệu quả nhất.

- **Symbolic execution** là cầu nối: chạy chương trình với input symbolic, tích lũy path constraint, gọi SMT giải.

Trong các bài tiếp theo, ta đi sâu vào hai thành phần của SMT: SAT (bài 3.4) và theory (bài 3.5).

Mini-quiz

Q1. Vì sao BFS cho counterexample ngắn nhất nhưng DFS không?

BFS khám phá state theo lớp: lớp 1 (depth 1), lớp 2 (depth 2), và cứ thế. Khi tới một bad state, đó là bad state đầu tiên trong thứ tự thăm, tức **gần initial state nhất** (depth nhỏ nhất). Counterexample là path từ initial tới bad, độ dài chính là depth.

DFS đi sâu vào một nhánh trước khi backtrack. Bad state đầu tiên gặp có thể nằm ở nhánh sâu mà DFS chọn ngẫu nhiên, không nhất thiết là bad state gần nhất. Ví dụ trong đồ thị nhỏ ở phần BFS, DFS có thể thăm S3, S4 (depth 2) trước S7 (depth 3), nhưng nếu bad state là S7, DFS vẫn cần đi qua S1, S3, S4 trước khi backtrack tới S2 $\square$ S5 $\square$ S7.

Q2. State explosion là gì? Cho một ví dụ định lượng.

State explosion là hiện tượng số state khả dĩ của chương trình tăng theo cấp số mũ với số biến. Ví dụ định lượng:

- 10 biến boolean: $2^{10} = 1024$ state.
- Thêm 1 int 32-bit: $1024 \times 2^{32} \approx 4 \times 10^{12}$ state.
- Thêm mảng 100 byte: $4 \times 10^{12} \times 2^{800} \approx$ vượt số nguyên tử trong vũ trụ.

Hệ quả: explicit-state model checking (liệt kê từng state) không scale được với chương trình thực tế. Phải dùng symbolic representation (BMC + SMT) hoặc abstract interpretation.

Q3. Symbolic execution và BMC khác nhau cụ thể ở điểm nào?

Cả hai đều dùng SMT solver, nhưng cách “đặt câu hỏi” khác nhau:

Symbolic execution chạy chương trình theo path, mỗi path tích lũy path constraint. Khi đến nhánh, fork thành nhiều world. Khi đến assertion, gọi solver với constraint của path hiện tại. Nhiều lần gọi solver (một lần mỗi path).

BMC encode toàn bộ chương trình (mọi path khả dĩ trong bound k) thành **một công thức duy nhất** Φ_k , rồi gọi solver một lần. Solver tự khám phá path nào dẫn tới counterexample.

Tradeoff: - SymExec dễ song song hoá (mỗi path là một task riêng), nhưng path explosion làm số task lớn. - BMC chỉ một query nhưng formula khổng lồ, solver phải khám phá nội bộ.

Trong thực tế, hai approach hội tụ: hầu hết tool BMC hiện đại có thể coi như “lazy symbolic execution” và ngược lại.

Tiếp theo: **3.4 SAT và DPLL**

3.4 SAT và thuật toán DPLL

Tóm tắt một dòng: SAT (Satisfiability) là bài toán “có tồn tại gán giá trị cho các biến boolean làm công thức đúng không?”. Mặc dù NP-complete về lý thuyết, các SAT solver hiện đại dùng thuật toán DPLL kết hợp Conflict-Driven Clause Learning (CDCL) đã giải được công thức có hàng triệu biến trong vài giây. Đây là engine cốt lõi mà BMC và SMT đều dựa vào.

Vì sao SAT quan trọng đến vậy?

Khi nói “SMT solver”, ta hay nghĩ tới một “hộp đen” giải mọi loại constraint. Thực ra hộp đen đó có một engine nhỏ hơn bên trong gọi là **SAT solver**, và mọi theory phức tạp (số nguyên, mảng, bitvector) cuối cùng đều được dịch về SAT để giải. Hiểu SAT là hiểu nền móng của BMC, SMT, và do đó cả Lecture 3 lẫn 4.

Có một câu chuyện thú vị về tầm quan trọng của SAT trong vài thập kỷ qua. Năm 1971, Stephen Cook chứng minh SAT là **NP-complete đầu tiên**, mở ra cả lĩnh vực complexity theory. Trong nhiều năm, người ta coi SAT là bài toán “khó”, không thực dụng. Đến những năm 1990, nhờ một số đột phá thuật toán (đặc biệt là CDCL), SAT solver bắt đầu giải được những instance “không tưởng”: công thức với hàng triệu biến, hàng chục triệu clause, trong vài phút trên laptop thông thường. Hiện tượng này gọi là “**SAT revolution**”.

Nhờ SAT revolution, một loạt ứng dụng trở nên khả thi: hardware verification, software verification (BMC), planning, scheduling, cryptanalysis (tìm collision trong hash), thậm chí giải Sudoku và Mario level generation. Tài liệu mà bạn đang đọc tồn tại được phần lớn nhờ vào SAT solver hiệu quả.

Trong bài này, ta sẽ đi qua:

1. Định nghĩa hình thức SAT và Conjunctive Normal Form (CNF).
2. Thuật toán DPLL gốc với unit propagation và pure literal.
3. Conflict-Driven Clause Learning (CDCL), cốt lõi của SAT solver hiện đại.
4. Vì sao CDCL nhanh đến vậy trong thực tế dù lý thuyết NP-complete.

Bài toán SAT chính thức

Cho một công thức boolean ϕ trên các biến $x_1, x_2, \dots, x_n$, **SAT** hỏi: có gán giá trị $\{0, 1\}^n$ cho các biến làm ϕ true không?

Nếu có, ta nói ϕ là **satisfiable** (SAT). SAT solver trả về gán cụ thể (gọi là **model**).

Nếu không, ta nói ϕ là **unsatisfiable** (UNSAT). SAT solver chứng minh không có model nào.

Ví dụ: - $\phi_1 = x \wedge \neg x$: UNSAT (mâu thuẫn nội tại). - $\phi_2 = x \vee y$: SAT, model $\{x = 1, y = 0\}$ (hoặc nhiều model khác). - $\phi_3 = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$: UNSAT (cả 4 case bị loại trừ).

Conjunctive Normal Form (CNF)

SAT solver thường yêu cầu input dưới dạng chuẩn gọi là **CNF**: công thức là **AND** của các **OR** của các **literal**, với literal là biến hoặc phủ định biến.

$$\phi = \bigwedge_{i=1}^m C_i, \quad C_i = \bigvee_j \ell_{i,j}$$

Mỗi C_i gọi là một **clause**. Mỗi $\ell_{i,j}$ là literal (như x_3 hoặc $\neg x_5$).

Ví dụ CNF:

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4) \wedge (x_2 \vee \neg x_4)$$

Vì sao CNF? Vì nó có cấu trúc đều đặn, dễ implement thuật toán. Mọi công thức boolean đều có thể chuyển sang CNF bằng quy trình **Tseitin transformation** (nhanh, kích thước tăng tuyến tính).

:::tip[Phép loại suy CNF] Hãy hình dung CNF như một danh sách yêu cầu. Mỗi clause là một yêu cầu mà ít nhất một literal trong đó phải đúng. Cả công thức đúng khi mọi yêu cầu được thoả.

Ví dụ “Tôi muốn một laptop có (Mac OR Linux) AND (≥ 16 GB RAM) AND (≥ 512 GB SSD)” là CNF với 3 clause. Một laptop chỉ “OK” khi thoả cả 3 clause. :::

Thuật toán DPLL: nền tảng

DPLL (Davis-Putnam-Logemann-Loveland, 1962) là thuật toán đầu tiên giải SAT một cách có hệ thống. Ý tưởng cơ bản: thử gán giá trị cho từng biến một, nếu mâu thuẫn thì quay lui (backtrack).

Pseudocode tổng quát:

```
function DPLL( $\phi$ , assignment):
     $\phi$  = unit_propagate( $\phi$ , assignment)
     $\phi$  = pure_literal_eliminate( $\phi$ , assignment)
    if  $\phi$  chứa empty clause:
        return UNSAT
    if mọi clause đều u satisfied:
        return SAT, assignment
    x = choose_variable( $\phi$ )
    return DPLL( $\phi \wedge x$ , assignment  $\cup \{x = 1\}$ )
        or DPLL( $\phi \wedge \neg x$ , assignment  $\cup \{x = 0\}$ )
```

Bốn ý tưởng then chốt:

1. Unit propagation

Unit clause là clause chỉ còn 1 literal chưa được gán. Ví dụ sau khi gán $x_1 = 0$, clause $(x_1 \vee x_2)$ trở thành (x_2) , vì $x_1 = 0$ không thoả được nên clause buộc $x_2 = 1$ mới đúng.

Unit propagation: khi gặp unit clause, **ép** giá trị literal đó. Nếu literal là x , gán $x = 1$. Nếu là $\neg x$, gán $x = 0$. Sau đó áp dụng lại trên các clause khác (có thể tạo thêm unit clause mới, “lan toà”).

Unit propagation là **lý do chính** SAT solver hiệu quả. Nó thường xử lý 80-90% các biến mà không cần đoán mò.

Ví dụ unit propagation

Cho công thức:

$$\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_4)$$

Thực hiện unit propagation:

- Clause $(\neg x_4)$ là unit, ép $x_4 = 0$.
- Áp vào $(\neg x_3 \vee x_4)$: $x_4 = 0$ không thoả, clause còn $(\neg x_3)$, là unit, ép $x_3 = 0$.
- Áp vào $(\neg x_1 \vee x_3)$: $x_3 = 0$ không thoả, clause còn $(\neg x_1)$, ép $x_1 = 0$.
- Áp vào $(x_1 \vee x_2)$: $x_1 = 0$ không thoả, clause còn (x_2) , ép $x_2 = 1$.

Kết quả: $\{x_1 = 0, x_2 = 1, x_3 = 0, x_4 = 0\}$. **Không cần đoán bất kỳ biến nào.** Toàn bộ 4 biến được suy ra từ unit propagation.

2. Pure literal elimination

Pure literal là literal chỉ xuất hiện ở một dạng (positive hoặc negative) trong toàn bộ công thức.

Ví dụ $\phi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_2 \vee x_3)$: x_1 xuất hiện 2 lần, đều positive $\square$ x_1 là pure positive literal.

Khi gán $x_1 = 1$, cả 2 clause chứa x_1 được thoả, ta loại bỏ chúng. Công thức nhỏ đi, dễ giải hơn.

Pure literal elimination ít hiệu quả hơn unit propagation và bị bỏ qua trong nhiều SAT solver hiện đại để tối ưu code path. Nhưng nó là một ý tưởng có giá trị về mặt giáo trình.

3. Branching (đoán mò)

Khi unit propagation và pure literal eliminate hết, nếu vẫn còn biến chưa gán và công thức chưa có empty clause, ta phải **đoán**. Chọn một biến x chưa gán, thử $x = 1$ trước. Nếu fail (dẫn tới UNSAT trong subtree), thử $x = 0$.

Chiến lược chọn biến (gọi là **decision heuristic**) ảnh hưởng rất lớn tới hiệu năng. Các heuristic phổ biến: - **VSIDS** (Variable State Independent Decaying Sum): biến nào xuất hiện trong nhiều conflict gần đây được ưu tiên. - **Jeroslow-Wang**: ưu tiên biến trong clause ngắn. - **MOMS** (Maximum Occurrences in Minimum Sized clauses): tương tự Jeroslow-Wang.

Trong SAT solver hiện đại, VSIDS chiếm ưu thế.

4. Backtracking

Khi đoán $x = 1$ dẫn tới UNSAT, quay lui (chronological backtracking), undo gán, thử $x = 0$.

DPLL gốc dùng chronological backtracking (luôn quay lui về biến gần nhất). CDCL (phần sau) dùng **non-chronological backtracking** thông minh hơn.

Ví dụ chạy DPLL từng bước

Cho công thức:

$$\phi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$$

DPLL chạy như sau:

Bước 1: không có unit clause, không có pure literal (mỗi biến xuất hiện cả positive và negative).

Bước 2: branching trên x_1 , thử $x_1 = 1$.

Áp vào công thức: - $(x_1 \vee x_2)$: thoả, bỏ. - $(\neg x_1 \vee x_2 \vee x_3)$: $\neg x_1 = 0$, còn $(x_2 \vee x_3)$. - $(\neg x_2 \vee \neg x_3)$: chưa thay đổi. - $(\neg x_1 \vee \neg x_2)$: $\neg x_1 = 0$, còn $(\neg x_2)$, là unit.

Bước 3: unit propagation $\neg x_2 \Rightarrow x_2 = 0$.

Áp tiếp: - $(x_2 \vee x_3): x_2 = 0$, còn (x_3) , là unit. - $(\neg x_2 \vee \neg x_3): \neg x_2 = 1$, thoả, bỏ.

Bước 4: unit propagation $x_3 \Rightarrow x_3 = 1$.

Mọi clause đã thoả. SAT với $\{x_1 = 1, x_2 = 0, x_3 = 1\}$.

Nếu thử $x_1 = 0$ ở bước 2 thay vì $x_1 = 1$: - $(x_1 \vee x_2): x_1 = 0$, còn (x_2) , unit, ép $x_2 = 1$. - $(\neg x_2 \vee \neg x_3): x_2 = 1, \neg x_2 = 0$, còn $(\neg x_3)$, ép $x_3 = 0$. - $(\neg x_1 \vee x_2 \vee x_3):$ thoả qua $\neg x_1 = 1$, bỏ. - $(\neg x_1 \vee \neg x_2):$ thoả qua $\neg x_1 = 1$, bỏ.

Cũng SAT với $\{x_1 = 0, x_2 = 1, x_3 = 0\}$. Một công thức có thể có nhiều model.

Conflict-Driven Clause Learning (CDCL)

DPLL gốc dùng chronological backtracking: khi gặp conflict, quay lui một bước, đảo giá trị, thử tiếp. Vấn đề: nếu conflict gốc nằm ở quyết định cách 50 biến trước, DPLL phải mò đi mò lại 50 lớp.

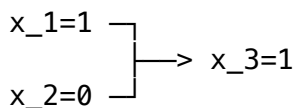
CDCL (1996, Marques-Silva và Sakallah) cải tiến bằng cách: khi gặp conflict, **phân tích** nguyên nhân, **học** một clause mới (gọi là **conflict clause**), và **quay lui xa hơn** tới đúng tầng quyết định gây ra conflict.

Bốn ý tưởng then chốt của CDCL:

Ý tưởng 1: Implication graph

Khi unit propagation chạy, ta vẽ một đồ thị có hướng: nút là literal đã gán, cạnh đi từ literal “nguyên nhân” tới literal “kết quả” (qua unit clause).

Ví dụ: nếu gán $x_1 = 1, x_2 = 0$, và clause $(\neg x_1 \vee x_2 \vee x_3)$ trở thành unit (x_3) , ép $x_3 = 1$, thì ta vẽ:



Đồ thị này tích lũy mọi gán ép từ unit propagation. Khi conflict xảy ra (một clause trở thành empty), ta có thể truy ngược tìm gốc.

Ý tưởng 2: Conflict analysis

Khi gặp empty clause, ta **truy ngược** implication graph để tìm tập các decision (gán mò) nào trực tiếp gây ra conflict. Tập này gọi là **conflict set**.

Phủ định conflict set tạo ra **conflict clause** (clause học được). Clause này thêm vào công thức, đảm bảo trong tương lai không bao giờ rơi vào cùng combination này nữa.

Ví dụ: nếu conflict do đồng thời $x_1 = 1, x_2 = 0, x_5 = 1$, conflict clause là $\neg x_1 \vee x_2 \vee \neg x_5$. Lần sau, bất kỳ đường nào đi tới $\{x_1 = 1, x_2 = 0, x_5 = 1\}$ đều bị clause này chặn ngay từ unit propagation.

Ý tưởng 3: Non-chronological backjumping

Sau khi học conflict clause, không quay lui 1 bước, mà quay lui tới **tầng quyết định cao nhất trong conflict set**. Ví dụ nếu conflict set có quyết định ở các tầng $\{1, 3, 5\}$, ta nhảy về tầng 3 (cao thứ hai), undo mọi gán sau đó, rồi tiếp tục với clause mới.

Backjumping có thể nhảy qua nhiều tầng cùng lúc, tiết kiệm rất nhiều công sức so với chronological backtracking.

Ý tưởng 4: Clause forgetting và restart

Càng học nhiều clause, bộ nhớ càng đầy và unit propagation càng chậm. Định kỳ, SAT solver **xóa bớt** các clause học được ít hữu ích (theo metric “clause activity”).

Restart: định kỳ xoá toàn bộ decision (nhưng giữ lại clauses học được), bắt đầu lại từ tầng 0. Lý do: heuristic chọn biến có thể đã rẽ sai từ đầu; restart cho cơ hội chọn lại với “kinh nghiệm” tích lũy trong các clause học được.

Vì sao CDCL nhanh đến vậy trong thực tế?

Lý thuyết: SAT là NP-complete. Worst case của CDCL vẫn là $O(2^n)$. Vậy tại sao thực tế nó giải được công thức triệu biến trong giây?

Câu trả lời nằm ở hai sự thật:

Sự thật 1: Hầu hết công thức SAT thực tế **không phải worst case**. Công thức từ hardware verification, software verification, planning đều có **cấu trúc** (biến và clause không độc lập, có liên kết logic). CDCL khai thác cấu trúc qua learning.

Sự thật 2: Mỗi conflict clause học được **cắt** một phần lớn không gian tìm kiếm. Sau hàng triệu conflict, không gian còn lại chỉ là một fraction rất nhỏ của 2^n .

Một analogy hữu ích: tìm kim trong đồng cỏ. Brute force là sờ từng cọng. CDCL là: mỗi lần sờ trúng “phần không có kim”, ghi nhớ và không bao giờ sờ vào phần đó nữa. Sau một triệu lần ghi nhớ, vùng phải tìm thu hẹp rất nhiều.

:::tip[Vì sao SAT competition quan trọng?] Mỗi năm, cộng đồng tổ chức **SAT Competition**: các solver thi giải cùng tập benchmark. Solver thắng cuộc thường được integrate vào BMC tool (CBMC, ESBMC) trong năm sau. Hiện tại (2024), các solver hàng đầu gồm Kissat, CaDiCaL, MapleSAT.

Nhờ cạnh tranh này, hiệu năng SAT solver tăng đều đặn ~2x mỗi vài năm trong 20 năm qua. Một benchmark “khó” của 2004 giờ giải được trong vài mili giây. :::

Mở rộng: SAT enabling cho các ngành khác

SAT solver mạnh mở ra ứng dụng cho rất nhiều lĩnh vực ngoài verification:

- **Hardware design:** equivalence checking giữa 2 thiết kế mạch.
- **Planning:** lập lịch tác vụ với nhiều ràng buộc.
- **Cryptanalysis:** tìm collision trong hash function, break stream cipher.
- **Bioinformatics:** phân tích pathway phản ứng hoá học.
- **Optimization:** kết hợp với MaxSAT cho bài toán tối ưu hoá có ràng buộc.

Có một cộng đồng nghiên cứu sôi động xung quanh SAT, với hội nghị thường niên SAT Conference, journal JSAT, và open dataset SATLIB.

Tóm tắt

- **SAT** là bài toán quyết định xem công thức boolean có thoả được không. NP-complete về lý thuyết.
- **CNF** là dạng chuẩn AND của OR, đầu vào tiêu chuẩn cho SAT solver.
- **DPLL** dùng unit propagation, pure literal, branching, backtracking.
- **CDCL** mở rộng DPLL với conflict analysis, learning, backjumping, restart.
- Hiệu năng thực tế của CDCL **vượt xa worst-case lý thuyết** nhờ khai thác cấu trúc và học conflict.

- SAT solver là engine cốt lõi cho mọi BMC tool (CBMC, ESBMC, SeaHorn) và SMT solver (Z3, CVC5, Yices).

Mini-quiz

Q1. Áp dụng unit propagation cho công thức $(\neg x_1) \wedge (x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$. Kết quả?

- Clause $(\neg x_1)$ là unit, ép $x_1 = 0$.
- Áp vào $(x_1 \vee x_2)$: $x_1 = 0$ không thoả, còn (x_2) , ép $x_2 = 1$.
- Áp vào $(\neg x_2 \vee x_3)$: $x_2 = 1, \neg x_2 = 0$ không thoả, còn (x_3) , ép $x_3 = 1$.

Kết quả: $\{x_1 = 0, x_2 = 1, x_3 = 1\}$. SAT, mọi clause thoả.

Q2. Pure literal là gì? Tại sao gán giá trị “thuận chiều” của pure literal luôn an toàn?

Pure literal là literal chỉ xuất hiện ở **một dạng** (positive hoặc negative) trong toàn bộ công thức. Ví dụ nếu x_1 chỉ xuất hiện positive (x_1 , không bao giờ $\neg x_1$), thì x_1 là pure positive.

Gán $x_1 = 1$ cho pure positive luôn an toàn vì: - Mọi clause chứa x_1 trở thành satisfied (bỏ được). - Không clause nào chứa $\neg x_1$ nên không thể tạo ra empty clause từ phép gán này.

Tức là gán pure literal không bao giờ làm công thức từ SAT thành UNSAT. Đây là một “safe move” thuần túy.

Q3. Vì sao CDCL được gọi là “non-chronological backtracking”?

DPLL gốc dùng chronological backtracking: khi gặp conflict ở tầng decision k , quay lui về tầng $k - 1$, đảo giá trị, thử lại.

CDCL phân tích conflict và xác định **tầng decision cao nhất trong conflict set** (gọi là tầng j , có thể $j \ll k - 1$). CDCL nhảy thẳng về tầng j , bỏ qua nhiều tầng trung gian. Đây là backjumping non-chronological, hay nhanh hơn chronological rất nhiều khi conflict gốc nằm ở quyết định xa.

Ví dụ: conflict ở tầng 100, conflict set chỉ có decision ở tầng 5 và 80. Chronological: backtrack 100 $\square$ 99 $\square$ 98 $\square$... mỗi lần lại bị mâu thuẫn. Non-chronological: nhảy thẳng từ 100 về 80, cùng với clause học được đảm bảo không lặp lại conflict.

Tiếp theo: **3.5 SMT theories**

3.5 SMT theories: Khi SAT không đủ

Tóm tắt một dòng: SMT solver mở rộng SAT bằng các **theory chuyên biệt** cho từng kiểu dữ liệu (số nguyên, bitvector, mảng, hàm). Kiến trúc DPLL(T) phối hợp SAT solver với “decision procedure” cho mỗi theory. Cho phép verifier diễn đạt và giải bài toán phức tạp hơn nhiều so với SAT thuần.

Vì sao SAT chưa đủ?

Ở bài 3.4, ta thấy SAT giải bài toán “công thức boolean có thoả không?”. Đó là một engine cực mạnh, nhưng có một hạn chế lớn: **chỉ làm việc với biến boolean**. Khi muốn diễn tả các ràng buộc phong phú hơn của chương trình, ta phải dịch về boolean, và bản dịch có thể rất dài.

Hãy thử ví dụ đơn giản. Bạn muốn check một property: “với $x, y \in \{0, 1, \dots, 100\}$, tồn tại nào làm $x + y > 150$ không?”. Để encode dưới SAT thuần, ta phải:

1. Biểu diễn x bằng 7 bit boolean (vì $100 < 128 = 2^7$): $x_0, x_1, \dots, x_6$.
2. Tương tự cho y .
3. Implement phép cộng $x + y$ bằng các cổng AND, OR, XOR (full adder), tạo ra ~30 biến boolean phụ.
4. So sánh kết quả với 150, dùng so sánh bitvector (thêm ~20 biến phụ).
5. Cuối cùng có công thức boolean ~60 biến, vài chục clause.

Vất vả! Hơn nữa, mọi lần thay đổi nhỏ (đổi 100 thành 1000) đều phải encode lại. Đây là cách tiếp cận **eager SMT** mà ta sẽ nói chi tiết bên dưới.

Cách thông minh hơn: cho phép ta viết trực tiếp $x + y > 150$ với x, y là số nguyên, và để solver tự xử lý phần arithmetic. Đó chính là **SMT solver**.

SMT = **S**atisfiability **M**odulo **T**heories. “Modulo” có nghĩa “với điều kiện kèm theo”: SMT kiểm tra satisfiability của công thức **modulo** các theory được tích hợp.

Các theory phổ biến

Mỗi theory cung cấp ngôn ngữ riêng để diễn đạt một loại constraint. Đây là các theory quan trọng nhất trong verification:

Theory of Equality and Uninterpreted Functions (EUF, UF)

Theory đơn giản nhất. Cho phép viết: - $(= a b)$: a bằng b . - $(f a)$: hàm f áp dụng vào a (nhưng không biết f là gì).

Tính chất quan trọng: nếu $(= a b)$, thì $(= (f a) (f b))$ đúng (functional consistency).

Ví dụ ràng buộc EUF:

```
(declare-sort U)
(declare-fun f (U) U)
(declare-const a U)
(declare-const b U)
(assert (= (f a) b))
(assert (= a (f b)))
(assert (not (= a b)))
```

Solver phải kiểm tra có nghiệm không. Phân tích: từ $(= (f\ a)\ b)$ và $(= a\ (f\ b))$, áp f cả hai vế của phương trình thứ hai: $(= (f\ a)\ (f\ (f\ b)))$. Kết hợp với phương trình đầu: $(= b\ (f\ (f\ b)))$. Kết hợp lần nữa: từ $(= (f\ a)\ b)$ có $(= (f\ (f\ b))\ (f\ b))$ (áp f vào hai vế $(= a\ (f\ b))$). Vậy $b = (f\ b)$. Tương tự dẫn tới $a = b$, mâu thuẫn với $(not\ (= a\ b))$. UNSAT.

EUF dùng cho verification của compiler optimization, abstract function call.

Linear Integer Arithmetic (LIA) và Linear Real Arithmetic (LRA)

Theory cho số nguyên (LIA) hoặc số thực (LRA), với các phép $+$, $-$, $\leq$, $<$, $=$ và nhân với hằng số.

Ví dụ:

```
(declare-const x Int)
(declare-const y Int)
(assert (>= x 0))
(assert (>= y 0))
(assert (= (+ x y) 100))
(assert (< (- x y) 10))
(check-sat)
```

Tìm $x, y \geq 0$ thỏa $x + y = 100$ và $x - y < 10$. Solver dùng thuật toán **Simplex** (giống Linear Programming) để giải.

LIA decidable nhưng tốn kém hơn LRA (do constraint nguyên). LRA decidable polynomial time với simplex.

:::warning[Linear vs Non-linear] “Linear” rất quan trọng. $(= z\ (*\ 2\ x))$ là linear (nhân với hằng số 2). $(= z\ (*\ x\ y))$ là **non-linear** (nhân hai biến). Non-linear integer arithmetic (NIA) là **undecidable** (Matiyasevich, 1970). Tool SMT có thể “thử” giải NIA nhưng không guarantee. :::

Bitvector Theory (BV)

Theory cho số bit-precise, mimic semantics của C/C++/Java. Đại diện số bằng dãy bit có độ dài cố định.

Ví dụ:

```
(declare-const x (_ BitVec 32))
(declare-const y (_ BitVec 32))
(assert (= y (bvmul x x)))
(assert (bvslt y #x00000000)) ; y < 0 trong signed
(check-sat)
```

Hỏi: có x nào làm $x \cdot x$ wrap thành số âm? Solver trả sat với $x = 46341$ (hoặc tương đương) cho $x^2 = 2147488281$, wrap về số âm 32-bit.

BV thực hiện qua **bit-blasting**: dịch mọi phép BV thành mạch boolean, gọi SAT solver giải. Đây là lý do BV thường chậm hơn LIA nhưng chính xác hơn cho semantics C. Chi tiết bit-blasting ở [bài 4.3](#).

Array Theory (A)

Theory cho mảng kích thước bất kỳ với hai phép cơ bản:

- `(select arr i)`: đọc phần tử thứ i .
- `(store arr i v)`: tạo mảng mới giống `arr` nhưng phần tử i thành v .

Tiên đề (axiom) quan trọng:

$$\text{select}(\text{store}(a, i, v), j) = \begin{cases} v & \text{nếu } i = j \\ \text{select}(a, j) & \text{nếu } i \neq j \end{cases}$$

Đây là axiom **read-over-write**: đọc một index ngay sau khi ghi vào index đó cho ra giá trị vừa ghi; đọc index khác cho ra giá trị cũ.

Array theory dùng để model **memory** trong chương trình C. Mọi $*p$, $a[i]$ đều dịch thành (select) . Mọi $*p = v$, $a[i] = v$ dịch thành (store) . Chi tiết encoding ở bài 3.7.

Floating-Point Theory (FP)

Theory cho số dấu phẩy động theo chuẩn IEEE 754. Hỗ trợ chính xác float, double, quad với mọi rounding mode, special value (NaN, $\pm\text{Inf}$, denormalized).

```
(declare-const x (_ FloatingPoint 8 24)) ; float 32-bit
(declare-const y (_ FloatingPoint 8 24))
(assert (= y (fp.add roundNearestTiesToEven x x)))
(assert (fp.isNaN y))
(check-sat)
```

Hỏi: có float x nào làm $x + x$ là NaN? Solver trả sat nếu $x = \text{NaN}$ (vì $\text{NaN} + \text{NaN} = \text{NaN}$).

FP theory rất chậm vì semantics phức tạp. Nhưng cần thiết cho verify code làm phép tính thực, ví dụ engine vật lý, ML inference. Chi tiết ở bài 3.6.

String Theory (S)

Theory cho chuỗi ký tự, hỗ trợ (str.contains) , (str.length) , (str.replace) , regex matching.

Dùng để verify code xử lý chuỗi: parser, sanitizer, regex matcher. Tool: Z3-str, CVC5.

Kiến trúc DPLL(T): cốt lõi SMT solver hiện đại

Câu hỏi: SMT solver kết hợp SAT solver với các theory như thế nào?

Có hai cách tiếp cận chính: **Eager SMT** và **Lazy SMT**. Hiện tại Lazy SMT (cụ thể là kiến trúc **DPLL(T)**) là chiếm ưu thế.

Eager SMT

Dịch toàn bộ công thức SMT thành công thức SAT trước, rồi gọi SAT solver một lần.

Ví dụ công thức $(x < 5) \text{ AND } (x > 3)$ với $x \in \mathbb{Z}$: - Eager: encode x thành bit vector 8-bit ($x_0, \dots, x_7$), encode $(x < 5)$ thành mạch so sánh bit, encode $(x > 3)$ tương tự, gọi SAT giải. - Pros: chỉ cần một SAT solver, đơn giản. - Cons: bit-blasting làm công thức cực lớn (cubic hoặc tệ hơn). Bay thông tin cấu trúc của theory.

Tool: Z3 mặc định dùng eager cho BV. Boolector hoàn toàn eager.

Lazy SMT với DPLL(T)

Tách công thức boolean (do SAT solver xử lý) khỏi phần theory (do **theory solver** xử lý). Phối hợp qua một giao diện chuẩn.

Hoạt động:

1. **Abstraction**: thay mỗi atom theory bằng một biến boolean. Ví dụ $(x < 5)$ được abstract thành biến p_1 . $(x > 3)$ thành p_2 . Công thức gốc $(x < 5) \text{ AND } (x > 3)$ thành công thức boolean $p_1 \text{ AND } p_2$.
2. **SAT solver**: giải công thức boolean. Nếu UNSAT, output UNSAT. Nếu SAT với model $\{p_1 = 1, p_2 = 1\}$, gửi cho theory solver.
3. **Theory check**: theory solver kiểm tra “có giá trị x làm p_1 true ($x < 5$) và p_2 true ($x > 3$) không?”. Có ($x = 4$). Output SAT.
4. **Theory conflict**: nếu theory solver nói “không có giá trị x nào thoả”, nó trả về **theory lemma** (một clause boolean ràng buộc các literal theory phải mâu thuẫn). Thêm lemma vào SAT, lặp lại.

Lazy SMT giống như SAT + plug-in cho từng theory. Mỗi theory chỉ cần implement một **decision procedure** trả lời “tập literal này có thoả được không?”.

DPLL(T): incremental version

DPLL(T) là Lazy SMT thực hiện theo cách **incremental**, gọi theory solver mỗi khi SAT solver gán thêm một literal, không đợi đến cuối. Như vậy theory conflict được phát hiện sớm, học conflict clause hiệu quả hơn.

Đây là kiến trúc của Z3, CVC5, Yices, MathSAT, hầu hết SMT solver thương mại và open source.

Nelson-Oppen: ghép nhiều theory

Câu hỏi tiếp: nếu công thức dùng cả LIA và Array, làm thế nào? Ví dụ:

```
(declare-const arr (Array Int Int))
(declare-const i Int)
(assert (= (select arr i) (+ i 1)))
(assert (= i 5))
(check-sat)
```

Phần $(= (\text{select arr } i) (+ i 1))$ cần cả Array theory và LIA: select thuộc Array, $+$ thuộc LIA.

Nelson-Oppen combination procedure (1979) là thuật toán ghép decision procedure của nhiều theory để có decision procedure tổng. Yêu cầu:

1. Mỗi theory phải có decision procedure riêng.
2. Các theory phải **disjoint** về signature (không chia sẻ symbol).
3. Theory phải **stably infinite** (có vô hạn model).

Thuật toán:

1. **Purify**: tách công thức thành các sub-formula, mỗi cái thuần một theory. Dùng biến intermediate để chia sẻ giá trị giữa các sub-formula.
2. **Equality propagation**: nếu một theory suy ra $x = y$, lan toả equality này sang các theory khác. Mỗi theory phải có khả năng phát hiện và “broadcast” equality.

3. **Conflict propagation**: nếu một theory phát hiện UNSAT, output UNSAT toàn cục.

Nelson-Oppen có tính chất “sound and complete” cho các theory thoả điều kiện. Đây là kỹ thuật cốt lõi đằng sau khả năng combo theory của Z3 và CVC5.

Ví dụ chạy DPLL(T) từng bước

Cho công thức SMT:

```
(declare-const x Int)
(declare-const y Int)
(assert (or (< x 0) (< y 0)))
(assert (> (+ x y) 5))
(assert (< x 3))
(assert (< y 4))
```

Hỏi: có x, y thoả tất cả không?

Bước 1: Abstract atom theory thành biến boolean: $-p_1 \sqcap (< x 0) - p_2 \sqcap (< y 0) - p_3 \sqcap (> (+ x y) 5) - p_4 \sqcap (< x 3) - p_5 \sqcap (< y 4)$

Công thức boolean: $(p_1 \vee p_2) \wedge p_3 \wedge p_4 \wedge p_5$.

Bước 2: SAT solver. Unit propagation cho p_3, p_4, p_5 thành true. Branching trên p_1, p_2 . Thử $p_1 = 1, p_2 = 0$.

Model boolean: $\{p_1 = 1, p_2 = 0, p_3 = 1, p_4 = 1, p_5 = 1\}$.

Bước 3: Theory check. Gửi cho LIA solver: $-p_1 = 1: x < 0$. $-p_2 = 0: \neg(y < 0)$, tức $y \geq 0$. $-p_3 = 1: x + y > 5$. $-p_4 = 1: x < 3$. $-p_5 = 1: y < 4$.

Có $x < 0, y \geq 0, x + y > 5, y < 4$. Từ $y < 4$ và $x + y > 5: x > 1$. Mâu thuẫn với $x < 0$.

LIA conflict: tập $\{p_1, \neg p_2, p_3, p_5\}$ mâu thuẫn. Theory lemma: $\neg p_1 \vee p_2 \vee \neg p_3 \vee \neg p_5$. Thêm vào SAT.

Bước 4: SAT solver tiếp tục với lemma mới. Buộc $p_1 = 0$ hoặc $p_2 = 1$. Thử $p_1 = 0, p_2 = 1$.

Theory check: $x \geq 0, y < 0, x + y > 5, x < 3, y < 4$. Từ $x < 3$ và $x + y > 5: y > 2$. Mâu thuẫn $y < 0$.

LIA conflict tương tự. Thêm lemma, SAT propagate, không có cách nào thoả cả $(p_1 \vee p_2)$ kèm các ràng buộc.

Output: UNSAT.

So sánh các SMT solver

Solver	Thế mạnh	Tốt cho
Z3 (Microsoft)	All-around mạnh, API tốt, license MIT	Verification, hardware, software
CVC5 (Stanford)	Strings, datatypes, separation logic	Web security, smart contract
Yices (SRI)	Rất nhanh cho QF_BV, LIA	Hardware verification
MathSAT (FBK)	Interpolation cho IC3	Model checking, IC3
Boolector (now Bitwuzla)	Pure BV, eager	Bitvector heavy

Trong BMC, CBMC dùng MiniSat (SAT), ESBMC dùng Z3 hoặc CVC5 (SMT).

Tóm tắt

- **SMT** mở rộng SAT bằng các **theory** cho data type chuyên biệt.
- **Theory phổ biến**: EUF, LIA, LRA, BV, Array, FP, String.
- **DPLL(T)** là kiến trúc Lazy SMT chiếm ưu thế: SAT solver phối hợp với theory solver theo incremental fashion.
- **Nelson-Oppen** ghép nhiều theory disjoint thành decision procedure tổng.
- BMC thường dùng SMT logic **QF_AUFBV** (Quantifier-Free Array UF Bitvector) cho C.

Mini-quiz

Q1. Vì sao gọi là “Modulo Theories”?

“Modulo” trong toán nghĩa là “với điều kiện kèm theo”. SMT kiểm tra satisfiability **với điều kiện** các theory đã được tích hợp đang hoạt động.

Ví dụ một công thức $(x < 5) \text{ AND } (x > 10)$ là UNSAT modulo LIA (theory số nguyên), vì LIA biết không có số nguyên nào vừa < 5 vừa > 10 . Nhưng nếu coi $<$ và $>$ chỉ là predicate boolean không có nghĩa, công thức trên SAT (gán cả hai true).

Nói cách khác, SMT solver kiểm tra “có model nào thoả công thức **và** thoả mọi tiên đề của các theory tích hợp không?”.

Q2. Phân biệt Eager SMT và Lazy SMT (DPLL(T)). Cái nào phổ biến hơn?

Eager SMT: dịch toàn bộ công thức SMT thành SAT formula trước (bit-blasting hoặc cách tương đương), rồi gọi SAT solver một lần. Đơn giản nhưng formula nổ về kích thước.

Lazy SMT (DPLL(T)): tách boolean (SAT solver) khỏi theory (theory solver). SAT giải phần boolean, theory solver kiểm tra theory constraint. Khi theory conflict, theory trả lemma về SAT.

DPLL(T) phổ biến hơn vì: - Không cần bit-blast tất cả (formula nhỏ hơn). - Có thể tận dụng decision procedure mạnh cho từng theory (Simplex cho LIA, congruence closure cho EUF). - Dễ extend với theory mới: chỉ cần implement decision procedure.

Eager vẫn dùng cho pure BV (Bitwuzla), khi theory không có decision procedure efficient.

Q3. Nelson-Oppen yêu cầu các theory phải “disjoint signature”. Vì sao?

“Disjoint signature” nghĩa là hai theory không dùng chung symbol. Ví dụ LIA dùng $+$, $-$, $<$ cho số nguyên, Array dùng `select`, `store` cho mảng. Không overlap.

Vì sao cần? Vì Nelson-Oppen hoạt động bằng cách **purify** công thức: chia thành sub-formula thuần một theory. Nếu hai theory dùng chung symbol (ví dụ cả LIA và LRA đều dùng $+$), thì khi gặp $(x + y)$ không biết là $+$ của LIA hay LRA.

Trong thực tế, các theory được thiết kế disjoint từ đầu. Khi cần kết hợp số nguyên với real, dùng theory mixed Int-Real có sẵn (LIRA) thay vì ghép LIA + LRA bằng Nelson-Oppen.

Tiếp theo: **3.6 Encoding numbers và floats**

3.6 Encoding numbers và floating-point

Tóm tắt một dòng: Để BMC verify chính xác semantics C, mọi kiểu số phải được encode đúng cách: số nguyên thành bitvector kích thước cố định, số thực thành IEEE 754 floating-point. Encoding sai cho kết quả sai (false negative hoặc false positive), vì thế bài này quan trọng cho việc dùng tool hiệu quả.

Vì sao bài này quan trọng?

Ở [bài 1.6](#), ta đã thấy một ví dụ kinh điển: hàm `abs(x)` được verify với theory `Int` thì pass, nhưng với theory `BitVec 32-bit` thì fail với input `INT_MIN`. Sự khác biệt này không phải lỗi của tool, mà phản ánh đúng sự khác biệt giữa **số nguyên toán học** và **số nguyên máy tính**.

Nếu bạn encode sai, hai kịch bản đáng sợ có thể xảy ra:

- **False negative** (miss bug): bạn dùng theory `Int` cho code C, tool trả “an toàn”, nhưng thực ra có bug overflow. Code đẩy production, bug phát nổ.
- **False positive** (báo nhầm): bạn over-approximate, tool báo bug nhưng input đó không khả thi trong môi trường thực. Developer mất công xem mỗi PR.

Bài này dạy bạn encode đúng cho mọi kiểu số chính của C: `char`, `short`, `int`, `long`, `unsigned`, `float`, `double`. Sau bài này, bạn có thể đọc output của CBMC/ESBMC và hiểu vì sao counterexample đó là counterexample thật.

Phần 1: Số nguyên với bitvector

Bitvector là gì?

Bitvector kích thước n là một dãy n bit có thứ tự. Trong SMT-LIB:

```
(declare-const x (_ BitVec 32))
```

Khai báo x là bitvector 32-bit. Giá trị của x là một số trong $\{0, 1, \dots, 2^{32} - 1\}$ nếu xem là unsigned, hoặc $\{-2^{31}, \dots, 2^{31} - 1\}$ nếu xem là signed (two's complement).

Điểm tinh tế: **bitvector tự nó không có dấu**. Nó chỉ là dãy bit. Việc “có dấu hay không” phụ thuộc vào **phép toán** bạn áp dụng. SMT-LIB cung cấp hai họ phép:

Phép	Unsigned	Signed
So sánh nhỏ hơn	<code>bvult</code>	<code>bvslt</code>
So sánh nhỏ hơn hoặc bằng	<code>bvule</code>	<code>bvsle</code>
Chia	<code>bvudiv</code>	<code>bvsdiv</code>
Modulo	<code>bvurem</code>	<code>bvsrem</code>

Còn cộng, trừ, nhân (`bvadd`, `bvsub`, `bvmul`), AND, OR, XOR (`bvand`, `bvor`, `bvxor`) không phân biệt dấu vì kết quả bit-level giống nhau.

Two's complement: lý thuyết và thực hành

Số có dấu trong máy tính dùng **two's complement**. Bit cao nhất (MSB) là dấu: 0 nghĩa dương, 1 nghĩa âm. Để chuyển từ số dương sang số âm tương ứng: đảo mọi bit rồi cộng 1.

Ví dụ với 8-bit: $-5_{10} = 00000101_2$ - -5 : đảo bit thành 11111010 , cộng 1 thành $11111011 = -5_{10}$

Two's complement có một tính chất rất hữu ích: phép cộng và trừ hoạt động **giống hệt** signed và unsigned ở mức bit. Phần cứng CPU không cần biết bạn đang dùng signed hay unsigned, chỉ cần biết cách check overflow.

Encode kiểu C sang bitvector

Bảng mapping (giả định x86-64):

Kiểu C	Bitvector	Phép so sánh dùng
char (signed mặc định trên GCC)	(<code>_ BitVec 8</code>)	<code>bvslt, bvsl</code>
unsigned char	(<code>_ BitVec 8</code>)	<code>bvult, bvule</code>
short	(<code>_ BitVec 16</code>)	<code>bvslt</code>
unsigned short	(<code>_ BitVec 16</code>)	<code>bvult</code>
int	(<code>_ BitVec 32</code>)	<code>bvslt</code>
unsigned int	(<code>_ BitVec 32</code>)	<code>bvult</code>
long	(<code>_ BitVec 64</code>)	<code>bvslt</code>
unsigned long, size_t	(<code>_ BitVec 64</code>)	<code>bvult</code>
bool	(<code>_ BitVec 1</code>)	<code>bvult</code>

Ví dụ: phát hiện integer overflow

Code C:

```
int add(int a, int b) {
    int c = a + b;
    assert(c >= a); // tin rằng "cộng s[] dương cho k[] t qu[] lớn hơn"
    return c;
}
```

SSA encoding:

```
a1 = nondet (BitVec 32)
b1 = nondet (BitVec 32)
c1 = bvadd(a1, b1)
assert(bvsge(c1, a1))
```

SMT-LIB với BV theory:

```
(set-logic QF_BV)
(declare-const a (_ BitVec 32))
(declare-const b (_ BitVec 32))
(declare-const c (_ BitVec 32))
(assert (= c (bvadd a b)))
(assert (not (bvsge c a)))
(check-sat)
(get-model)
```

Solver trả sat với model như: $a = 1$, $b = \$INT_MAX$. Khi $a + b = 1 + (2^{31} - 1) = 2^{31}$ wrap về -2^{31} , nhỏ hơn $a = 1$, assertion fail.

Bug này không thể bắt được nếu encode bằng theory Int (vì trong toán, $a + b > a$ khi $b > 0$).

Phát hiện signed/unsigned confusion

Một dạng bug rất tinh tế trong C: dùng signed comparison với giá trị thực sự là unsigned (hoặc ngược lại).

```
int receive(unsigned int len) {
    char buf[256];
    if (len > sizeof(buf)) return -1; // check unsigned: OK
    memcpy(buf, src, len);           // copy len bytes
    return 0;
}
```

Hàm này có vẻ an toàn. Nhưng nếu caller gọi `receive(-1)`?

```
receive(-1); // -1 cast sang unsigned int = 0xFFFFFFFF = 4294967295
```

Trong hàm `receive`, `len = 4294967295`. `len > 256` đúng, hàm `return -1`, an toàn. Vậy không phải bug.

Nhưng nếu code check như sau (lỗi tinh tế):

```
int receive_bad(int len) { // KIỂU SIGNED!
    char buf[256];
    if (len > sizeof(buf)) return -1; // signed > unsigned: tricky!
    memcpy(buf, src, len);
    return 0;
}
```

C standard nói gì? Khi so sánh `int len` với `sizeof(buf)` (kiểu `size_t`, unsigned), compiler **promote** `len` sang `size_t`. Nếu `len = -1`, promote thành `0xFFFFFFFF`. `0xFFFFFFFF > 256` đúng, `return -1`.

Nhưng dấu check không đủ. Nếu `len = -100`, promote thành `0xFFFFF9C` (xấp xỉ $2^{32} - 100$), vẫn `> 256`, `return -1`.

Vậy bug ở đâu? Ở `memcpy(buf, src, len)` mà `len` có kiểu `int`. Khi truyền cho `memcpy` (yêu cầu `size_t`), `-1` cast thành `0xFFFFFFFF`, `memcpy` copy $2^{32} - 1$ byte. Buffer overflow khổng lồ.

BMC với BV theory phát hiện được bug này nếu ta thêm `assert len >= 0`:

```
(set-logic QF_BV)
(declare-const len (_ BitVec 32)) ; signed int
(declare-const buf_size (_ BitVec 64)) ; size_t
(assert (= buf_size #x00000000000000100)) ; 256
(declare-const len_promoted (_ BitVec 64))
(assert (= len_promoted ((_ sign_extend 32) len))) ; sign-extend khi promote
(assert (bvugt len_promoted buf_size)) ; condition c[] a if
(assert (bvslt len #x00000000)) ; len thực ra âm
(check-sat)
```

Solver trả sat với `len = -1`. Branching không trigger, bug ở `memcpy` không check signed.

Bitvector encoding cho phép phức tạp

Một số phép C trông đơn giản nhưng encoding tinh tế:

Division by zero: `bvudiv` và `bvsdiv` không định nghĩa khi chia 0. SMT-LIB chuẩn 2.6 trả về `(_ bv0 N)` hoặc all-ones. Tool BMC thường thêm assertion `denominator != 0` ngay trước phép chia.

Shift count overflow: `bvshl x n` với $n \geq 32$ (cho BitVec 32) là Undefined Behavior trong C. BMC thường model bằng `(ite (bvugt n 31) (_ bv0 32) (bvshl x n))`.

Sign extension và zero extension: `((_ sign_extend N) x)`: mở rộng `x` từ kích thước nhỏ sang lớn, giữ dấu (copy MSB). `((_ zero_extend N) x)`: mở rộng, fill 0 ở các bit cao. Tương ứng `cast int → long (sign)` và `unsigned → unsigned long (zero)`.

Phần 2: Floating-point với IEEE 754

IEEE 754: cấu trúc bit của float

Số float 32-bit (single precision) trong IEEE 754:

1 bit	8 bits	23 bits
sign	exponent	mantissa (fraction)

Công thức:

$$\text{value} = (-1)^{\text{sign}} \times 2^{\text{exponent}-127} \times (1.\text{mantissa})_2$$

Ví dụ: $-1.0 = 0 \mid 01111111 \mid 000000000000000000000000 = 0x3F800000$. $-2.0 = 1 \mid 10000000 \mid 000000000000000000000000 = 0xC0000000$.

Có các giá trị đặc biệt:

Giá trị	Sign	Exponent	Mantissa
+0	0	0	0
-0	1	0	0
$+\infty$	0	255	0
$-\infty$	1	255	0
NaN	any	255	$\neq 0$
Denormalized (rất nhỏ)	any	0	$\neq 0$

Double precision (double, 64-bit): 1 sign + 11 exponent + 52 mantissa.

Vì sao float khó verify?

Float có nhiều property khó tin: $-0.1 + 0.2 \neq 0.3$: Vì 0.1 và 0.2 không biểu diễn chính xác trong nhị phân (chu kỳ vô hạn). Kết quả là 0.30000000000000004. - **NaN** $\neq$ **NaN**: Phép so sánh `nan == nan` trả về false. Đây là rule của IEEE 754. - **Rounding mode** ảnh hưởng kết quả: $(a + b) + c$ có thể khác $a + (b + c)$ vì rounding tích lũy. - **Subnormal numbers** (gần 0) có precision giảm dần.

Mọi sự lạ này phải được encode chính xác trong SMT để verifier đúng.

Theory FP trong SMT-LIB

```
(declare-const x (_ FloatingPoint 8 24)) ; single precision
(declare-const y (_ FloatingPoint 11 53)) ; double precision

; Phép: cộng với rounding mode
(assert (= z (fp.add roundNearestTiesToEven x y)))

; Predicate
(assert (fp.isNaN x))
(assert (fp.isInfinite y))
(assert (fp.eq x y)) ; equality theo IEEE 754 (NaN != NaN!)
```

Các rounding mode: - roundNearestTiesToEven (default, IEEE 754 “round half to even”). - roundNearestTiesToAway. - roundTowardPositive. - roundTowardNegative. - roundTowardZero (truncate).

Ví dụ verify: float associativity

Code C:

```
float assoc(float a, float b, float c) {
    float r1 = (a + b) + c;
    float r2 = a + (b + c);
    assert(r1 == r2); // tin rằng cộng có tính ch⊔ t k⊔ t hợp
    return r1;
}
```

Trong toán, $(a + b) + c = a + (b + c)$ luôn đúng. Trong float, **không**.

Encode SMT:

```
(set-logic QF_FP)
(declare-const a (_ FloatingPoint 8 24))
(declare-const b (_ FloatingPoint 8 24))
(declare-const c (_ FloatingPoint 8 24))
(declare-const r1 (_ FloatingPoint 8 24))
(declare-const r2 (_ FloatingPoint 8 24))
(assert (= r1 (fp.add roundNearestTiesToEven (fp.add roundNearestTiesToEven a b) c)))
(assert (= r2 (fp.add roundNearestTiesToEven a (fp.add roundNearestTiesToEven b c))))
(assert (not (fp.eq r1 r2)))
(check-sat)
(get-model)
```

Solver trả sat với counterexample như $a = 10^{20}$, $b = -10^{20}$, $c = 1$: $-(a + b) + c = 0 + 1 = 1$. $-a + (b + c) = 10^{20} + (-10^{20} + 1) = 10^{20} + (-10^{20}) = 0$ (vì 1 quá nhỏ để ảnh hưởng tới 10^{20}).

Hai kết quả khác nhau, dù toán học nói chúng bằng nhau.

Casts giữa float và int

Phép float $f = (\text{float}) i$; $(\text{int} \rightarrow \text{float})$ và int $i = (\text{int}) f$; $(\text{float} \rightarrow \text{int})$ cần encoding tinh tế:


```

; int → float
(declare-const i (_ BitVec 32))
(declare-const f (_ FloatingPoint 8 24))
(assert (= f ((_ to_fp 8 24) roundNearestTiesToEven i)))

; float → int (truncate toward zero)
(declare-const g (_ FloatingPoint 8 24))
(declare-const j (_ BitVec 32))
(assert (= j ((_ fp.to_sbv 32) roundTowardZero g)))

```

Cast float $\rightarrow$ int có thể tạo Undefined Behavior nếu float quá lớn (vượt INT_MAX) hoặc là NaN. C standard không định nghĩa kết quả. BMC tool thường thêm assertion để báo lỗi.

Tại sao tool BMC mặc định không dùng FP?

Float verification rất chậm vì:

1. FP theory phức tạp, decision procedure không đơn giản như LIA.
2. Eager bit-blasting cần $\sim 40,000$ gate cho một phép fp.add single precision.
3. Nhiều bug FP nằm ở subnormal hay rounding biên, cần khám phá kỹ.

Vì thế CBMC, ESBMC mặc định **tắt** FP support trừ khi user bật `--floatbv`. Khi tắt, tool model float như uninterpreted function: chỉ check khái niệm “không chia cho 0” mà không check chính xác giá trị.

Nếu bạn verify code numeric (engine physics, ML inference), bật FP support. Nếu chỉ verify control flow của code Web/CRUD, không cần.

Tóm tắt

- **Bitvector** là dãy bit có kích thước cố định. Phép có hai họ signed/unsigned.
- Encode kiểu C: int $\rightarrow$ 32-bit BV, long $\rightarrow$ 64-bit BV, unsigned X $\rightarrow$ BV cùng size dùng phép unsigned.
- BMC + BV bắt được integer overflow, sign/unsigned confusion, shift overflow.
- **FP theory** model IEEE 754 chính xác, kể cả NaN, Inf, denormalized, rounding mode.
- FP verification chậm; chỉ bật khi code numeric.

Mini-quiz

Q1. Vì sao theory Int (toán học) cho kết quả khác BitVec 32-bit cho cùng chương trình C?

Theory Int model số nguyên toán học vô hạn dài. Theory BitVec 32-bit model số trong dải $[-2^{31}, 2^{31} - 1]$ (signed) hoặc $[0, 2^{32} - 1]$ (unsigned), với wrap-around khi tràn.

Code C dùng số 32-bit, có wrap-around. Verify với Int **bỏ qua** mọi bug overflow. Verify với BV mới bắt được.

Ví dụ int `x = INT_MAX; x = x + 1; assert(x > 0);` - Int: $x = 2^{31} - 1 + 1 = 2^{31}, > 0$, assert pass. - BV: `x` wrap về $-2^{31}, < 0$, assert fail.

BMC tool mặc định dùng BV cho C để khớp semantics thực.

Q2. Vì sao `nan == nan` trả về false trong IEEE 754?

Theo IEEE 754, NaN biểu diễn “kết quả không xác định” (ví dụ $0/0$, $\sqrt{-1}$ với real, $\log(-1)$). Hai NaN có thể đến từ hai phép khác nhau, mang ý nghĩa khác nhau. Tiêu chuẩn quyết định **mọi phép so sánh có NaN đều trả false**, bao gồm `nan == nan`.

Hệ quả thực dụng: để check `x` có phải NaN không, không dùng `x == nan`. Dùng `isnan(x)` hoặc tự check `x != x` (chỉ NaN cho true).

Trong SMT theory FP, `(fp.eq x y)` là IEEE 754 equality (NaN != NaN). Nếu muốn “bitwise equality” (xem hai bit pattern có giống không, kể cả NaN), dùng `(= x y)` direct.

Q3. Phân biệt `bvslt` và `bvult`. Cho ví dụ khi chúng cho kết quả khác nhau.

- `bvslt`: signed less than. Diễn giải bitvector là số two’s complement.
- `bvult`: unsigned less than. Diễn giải bitvector là số nguyên không âm.

Ví dụ với BitVec 8-bit, $x = 0xFF$, $y = 0x01$: - `bvslt x y`: signed, $x = -1$, $y = 1$. $-1 < 1$, **true**. - `bvult x y`: unsigned, $x = 255$, $y = 1$. $255 < 1$, **false**.

Cùng bit pattern, kết quả ngược nhau. Đây là nguồn của nhiều bug “signed/unsigned confusion” trong C. Encoding đúng giúp BMC phát hiện ra.

Tiếp theo: [3.7 Encoding pointers và memory](#)

3.7 Encoding pointers và memory: phần khó nhất

Tóm tắt một dòng: Memory trong C là chỗ khó nhất để encode vì pointer có thể trỏ tới bất cứ đâu, hai pointer có thể alias (cùng trỏ một địa chỉ), và allocator động (malloc) tạo ra object mới tại runtime. BMC tool dùng **array theory** kết hợp với **alias analysis** để model memory một cách tractable nhưng vẫn chính xác.

Vì sao memory khó?

Hãy nhìn vào một đoạn code C đơn giản:

```
int x = 5;
int y = 10;
int *p = &x;
int *q = (rand() % 2) ? &x : &y;
*p = 100;
assert(*q == 100 || *q == 10);
```

Câu hỏi: assertion đúng không?

Trực giác: *q là x (= 100 sau gán) hoặc y (= 10). Cả hai đều thoả assertion.

Đúng. Nhưng để **chứng minh** điều này một cách tự động, BMC tool phải:

1. Biết &x và &y là hai địa chỉ **khác nhau**.
2. Biết p chắc chắn trỏ tới x.
3. Biết q có thể trỏ tới x hoặc y (do rand()).
4. Biết khi gán *p = 100, chỉ giá trị tại địa chỉ &x thay đổi, không phải &y.
5. Suy ra *q là 100 (nếu q = &x) hoặc 10 (nếu q = &y).

Điểm 4 là khó nhất. Trong C, gán *p = 100 chỉ thay đổi 1 word memory. Encoding sai (ví dụ “thay đổi toàn bộ memory”) cho false negative; encoding chính xác đòi hỏi biết *p chính xác đến đâu, tức **alias analysis**.

Bài này dạy cách BMC tool giải quyết các vấn đề trên.

Phần 1: Array theory để model memory

Ý tưởng cơ bản

Trong array theory: - (select arr i) đọc phần tử thứ i. - (store arr i v) tạo mảng mới giống arr nhưng phần tử i là v.

Áp dụng cho memory: - Mọi memory là một mảng huge: memory: Array (BitVec 64) (BitVec 8). Index là địa chỉ 64-bit, value là byte. - Đọc memory: (select memory addr) cho byte tại địa chỉ addr. - Ghi memory: (store memory addr value) cập nhật byte.

Mỗi gán trong C tạo ra một version mới của memory (đúng SSA spirit):

```
*p = 100;
```

Thành:

```
memory_2 = store(memory_1, addr(p), 100_as_4_bytes)
```

Read-over-write axiom: nếu đọc địa chỉ vừa ghi, được giá trị mới. Đọc địa chỉ khác, giữ nguyên giá trị cũ. Đây chính là cách array theory **suy luận về aliasing** tự nhiên.

Multi-byte read/write

Memory là array của **byte**, nhưng C có `int` (4 byte), `long` (8 byte), `double` (8 byte). Đọc một `int` từ memory cần:

```
read_int(memory, addr) = concat(
  select(memory, addr+0),
  select(memory, addr+1),
  select(memory, addr+2),
  select(memory, addr+3)
)
```

Tương tự cho ghi:

```
write_int(memory, addr, value) =
  store(store(store(store(memory,
    addr+0, value[7:0]),
    addr+1, value[15:8]),
    addr+2, value[23:16]),
    addr+3, value[31:24])
```

(Giả sử little-endian như x86.)

Encoding 4 store/select cho mỗi `int` truy cập làm formula lớn. Optimization: dùng **multi-byte array theory** (mảng `BitVec 64` → `BitVec 32` hoặc `BitVec 64` → `BitVec 8`) tùy tool.

Phần 2: Encoding pointer

Pointer là gì trong logic?

Trong C, pointer là một địa chỉ 64-bit (trên x86-64). Tự nhiên ta encode bằng:

```
(declare-const p (_ BitVec 64))
```

Đọc qua pointer: `*p` thành `read_int(memory, p)`.

Nhưng có một vấn đề. Trong C, pointer còn mang thông tin về **object** mà nó trỏ tới, không chỉ địa chỉ. Ví dụ:

```
int arr[10];
int *p = arr;
p += 100;    // out of bounds, nhưng C standard nói UB
*p = 5;      // UB!
```

Code này có UB vì `p` trỏ ngoài `arr`. Nhưng nếu chỉ encode bằng địa chỉ thuần, BMC có thể “miss” vì `p + 400` vẫn là một địa chỉ hợp lệ trong memory map.

Để giải quyết, một số tool (ESBMC, CBMC modern) dùng **two-level encoding** cho pointer:

```
(declare-datatype Pointer (
  (mk-ptr (object Int) (offset (_ BitVec 64)))
))
```

Pointer là một cặp (object_id, offset). object_id là ID duy nhất của object (mỗi int x có ID khác, mỗi malloc có ID khác). offset là vị trí trong object.

Khi `p += 100`, offset tăng nhưng object_id giữ nguyên. Khi dereference, tool check offset có nằm trong bound của object không, báo lỗi nếu vượt.

Two-level memory model của ESBMC

ESBMC dùng một biến thể của hai cấp:

- **Object table**: mỗi object có entry chứa (object_id, size, address_base).
- **Memory array**: như trên, nhưng indexed bởi (object_id, offset) thay vì địa chỉ thuần.

Cấu trúc này cho phép: - Detect out-of-bounds tự động (offset >= size). - Aliasing analysis chính xác: hai pointer alias khi và chỉ khi cùng object_id và cùng offset. - Symbolic pointer arithmetic dễ hơn.

Nhược điểm: model phức tạp hơn, query SMT chậm hơn so với “flat memory” thuần.

Phần 3: Malloc và động học bộ nhớ

Encode malloc

`malloc(n)` tạo một object mới có kích thước n byte. Trong BMC:

```
int *p = malloc(40);
```

Thành:

```
new_object_id = unique_fresh_id()
p = mk_ptr(new_object_id, 0)
object_table = extend(object_table, new_object_id, size=40, base=fresh_addr)
```

Khi `free(p)`:

```
mark_freed(object_table, object(p))
```

Mọi access sau free phải báo “use after free” assertion.

Detect use-after-free

```
int *p = malloc(40);
*p = 5;
free(p);
*p = 10;    // use after free!
```

Encode:

```
obj1 = fresh_id()
p = (obj1, 0)
memory_1 = store(memory_0, &(obj1, 0), 5_4bytes)
freed_1 = set_add(freed_0, obj1)
assert(not (obj1 in freed_1)) ; check trước khi write
memory_2 = store(memory_1, &(obj1, 0), 10_4bytes)
```

Assertion fail tại bước check, BMC trả counterexample.

Detect double free

```
int *p = malloc(40);
free(p);
free(p);    // double free!
```

Encode:

```
freed_1 = set_add(freed_0, obj1)
assert(not (obj1 in freed_1)) ; trước free thứ hai
freed_2 = set_add(freed_1, obj1)
```

Assertion fail.

Detect memory leak

Leak là khi một object được allocate nhưng không free trước khi pointer cuối cùng trở tới nó bị overwrite/out-of-scope. Detect leak khó hơn vì cần “garbage collector” trong BMC, không phải lúc nào cũng làm.

CBMC có flag `--memory-leak-check` để bật leak detection. ESBMC tương tự.

Phần 4: Aliasing analysis

Vì sao aliasing khó?

Hai pointer **alias** khi cùng trỏ một địa chỉ. Aliasing quan trọng vì:

```
void f(int *p, int *q) {
    *p = 5;
    *q = 10;
    int x = *p;    // x = 10 (nếu p, q alias) hoặc 5 (nếu không)
}
```

Tool cần biết p và q có alias không. Có 3 case: - **Must-alias**: chắc chắn alias (ví dụ $q = p$ trước đó). - **No-alias**: chắc chắn không alias (ví dụ $\&x$ và $\&y$ của 2 local var khác nhau). - **May-alias**: có thể alias, có thể không (ví dụ pointer từ function call).

Anderson và Steensgaard

Hai thuật toán cơ bản:

Andersen analysis (inclusion-based): mỗi pointer có một “points-to set”. Khi $p = \&x$, thêm x vào set của p. Khi $q = p$, set của q chứa set của p. Phân tích chính xác hơn nhưng $O(n^3)$.

Steensgaard analysis (union-based): mỗi assignment hợp nhất các points-to set. Nhanh hơn ($O(n \cdot \alpha(n))$) nhưng kém chính xác.

Tool BMC thường dùng aliasing analysis ở pre-processing để giảm encoding size, rồi để SMT solver xử lý phần residual.

Encode alias precisely với array theory

Nếu không pre-analyze, BMC vẫn handle alias chính xác qua array theory. Ví dụ:

```
int x = 5, y = 10;
int *p = &x;
int *q = (cond) ? &x : &y;
*p = 100;
int r = *q;
```

Encode:

```
mem_0: ban đầ u, có x = 5 tại addr_x, y = 10 tại addr_y
p = addr_x
q = ite(cond, addr_x, addr_y)
mem_1 = store(mem_0, p, 100) = store(mem_0, addr_x, 100)
r = select(mem_1, q) = select(store(mem_0, addr_x, 100), q)
```

Áp dụng read-over-write:

```
r = ite(q == addr_x, 100, select(mem_0, q))
  = ite(cond, 100, ite(addr_y == addr_x, 100, 10)) ; q expand
  = ite(cond, 100, 10) ; addr_y != addr_x
```

Kết quả: $r = 100$ nếu cond true, $r = 10$ nếu false. Chính xác.

Array theory tự động handle aliasing qua axiom read-over-write. Không cần aliasing analysis riêng, dù pre-analysis làm encoding nhanh hơn.

Phần 5: Alignment và padding

C compiler chèn padding để align field của struct. Ví dụ:

```
struct S {
    char c;      // offset 0
    int i;       // offset 4 (padding 3 byte ở 1-3)
};
```

`sizeof(struct S) = 8`, không phải 5.

BMC tool phải model padding chính xác. Đọc `c` cho byte 0. Đọc `i` cho byte 4-7. Byte 1-3 là padding (giá trị không xác định).

Nếu encoding sai (giả định không padding), BMC có thể miss bug. Ví dụ:

```
struct S s;
s.c = 'A';
s.i = 0x12345678;
char *p = (char*) &s;
char x = p[2]; // đọc padding byte 2 (UB!)
```

`x` là byte padding, không xác định. Code có UB. BMC chuẩn (như ESBMC với `--memory-model fixed`) phát hiện và báo “non-deterministic value access”.

Phần 6: ESBMC memory model trong thực tế

ESBMC cung cấp ba memory model qua flag:

Flag	Memory model	Speed	Precision
<code>--memory-model fixed</code>	Flat byte array với layout C cố định	Trung bình	Cao
<code>--memory-model align</code>	Aligned-only access	Nhanh hơn	Trung bình
<code>--memory-model offset</code>	Object-offset two-level	Chậm hơn	Cao nhất

Cho production code, `fixed` là default tốt. Cho code có nhiều pointer arithmetic phức tạp (kernel driver, allocator), `offset` chính xác hơn.

CBMC cũng có thiết kế tương tự với flags `--object-bits N` (số bit dùng cho `object_id`).

Tóm tắt

- **Array theory** model memory như mảng huge từ địa chỉ (BitVec 64) sang byte (BitVec 8).
- Multi-byte read/write phân rã thành nhiều `select/store`.
- **Pointer** encode bằng cặp (`object_id`, `offset`) để track object identity, phát hiện out-of-bounds.
- **Malloc/free/leak** model qua `object_table`, `freed_set`.
- **Aliasing**: array theory tự handle qua read-over-write; pre-analysis (Andersen, Steensgaard) tăng tốc.
- **Padding và alignment** phải model chính xác để khớp semantics C.
- ESBMC có ba memory model: `fixed` (default), `align`, `offset`.

Mini-quiz

Q1. Vì sao gọi memory model là “two-level”?

Một cấp là **địa chỉ tuyến tính** (số 64-bit). Cấp khác là **object identity + offset** (cặp).

Cấp địa chỉ tuyến tính tự nhiên với phần cứng (CPU thực sự dùng địa chỉ flat). Cấp object-offset tự nhiên với semantics C (mỗi `malloc`, mỗi local var là một object riêng).

Two-level encoding kết hợp cả hai: pointer là cặp (`object_id`, `offset`), có thể compute thành địa chỉ tuyến tính khi cần. Tool BMC dùng object identity để track bounds, dùng địa chỉ tuyến tính để model pointer arithmetic across object (UB nhưng vẫn phải model).

Q2. Read-over-write axiom giúp giải quyết aliasing như thế nào?

Read-over-write: `select(store(arr, i, v), j)` bằng `v` nếu `i == j`, bằng `select(arr, j)` nếu `i != j`.

Khi BMC encode hai pointer access `*p` và `*q`, nó không cần biết trước `p` và `q` có alias không. Nó encode:

```
mem_1 = store(mem_0, p, val_p)    ; gán *p
result = select(mem_1, q)          ; đọc *q
        = ite(p == q, val_p, select(mem_0, q))
```

SMT solver sau đó tự suy luận: liệu `p == q` (alias) hay không. Nếu solver chứng minh được `p != q`, simplify thành `select(mem_0, q)`. Nếu không chắc, giữ `ite`, để alias là biến boolean.

Cách này **chính xác**: bắt được mọi case alias mà không cần aliasing analysis riêng. Đánh đổi: formula lớn, solver tốn công.

Q3. Tại sao detect memory leak khó hơn detect use-after-free?

Use-after-free là property local: tại mỗi access, check object đã free chưa. Đơn giản encode bằng `freed_set`.

Memory leak là property global: tại điểm chương trình kết thúc, mọi allocated object phải đã free hoặc còn pointer trỏ tới. Cần: 1. Track mọi allocation (đã có). 2. Track mọi pointer hiện đang trỏ tới mỗi object (khó: pointer copy, store vào struct, pass qua function). 3. Tại exit, check object nào không còn pointer trỏ tới nó.

Bước 2 yêu cầu **escape analysis** hoặc **reachability** từ root set (global var, stack var). Đây là phân tích đắt.

CBMC và ESBMC có `--memory-leak-check` nhưng chậm hơn nhiều và đôi khi không sound (miss leak hoặc báo nhầm) trong code phức tạp.

Kết thúc Cụm 2 (Lecture 3). Bạn đã hiểu sâu cách BMC + SMT thực sự hoạt động bên trong. Chuyển sang **Lecture 4: Static Analysis II (Concurrency)** để xem cách xử lý chương trình đa luồng.